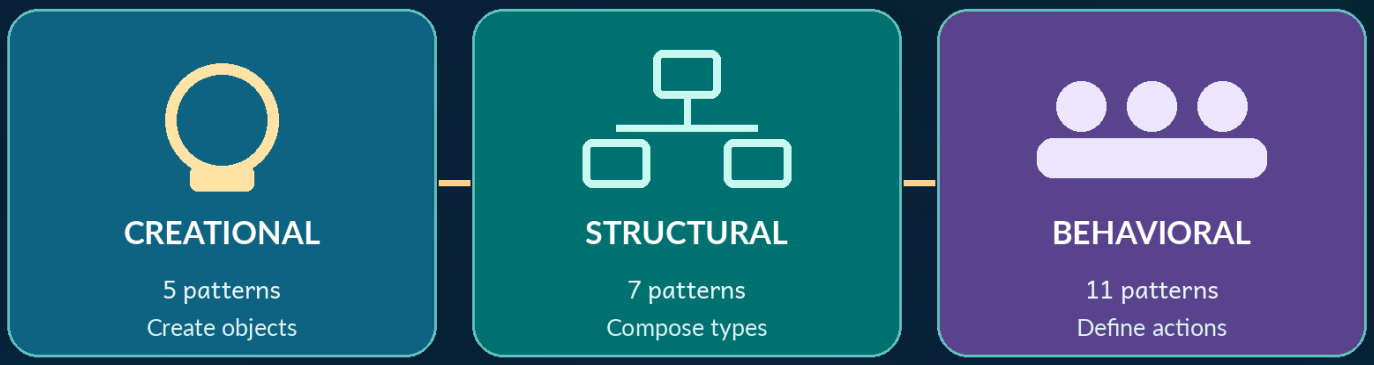
THE PREMIUM EDITION

JAVA DESIGN PATTERNS

ZERO TO HERO INTERVIEW GUIDE

Premium handwritten-style study notes



100 INTERVIEW QUESTIONS

23 GOF PATTERNS | 12 JAVA LABS

28 CHAPTERS | SPRING EXAMPLES | 30-DAY PLAN

Java 17+ / SOLID / UML ROLE DIAGRAMS / SENIOR SCENARIOS

The complete interview-preparation collection

THE LEARNING SYSTEM

Learn the intent. Explain the trade-off.

This edition keeps the 100 original interview question wordings. Read the quick answer, study the original example, trace the role diagram, then answer the harder follow-up without looking.

01

Understand the problem

What changes? Which object owns the responsibility?

02

Explain the pattern

Describe the collaboration, not only a class definition.

03

Trace the example

Read the small Java snippet or role diagram.

04

Name the risk

Thread safety, mutation, identity, failure or hidden coupling.

05

Answer the follow-up

Defend a test or production scenario in simple English.

Study baseline: Java 17+ standard-library labs. Spring framework snippets are conceptual and require Spring dependencies.

GUIDED TABLE OF CONTENTS

28 topic chapters

Original interview question sequence retained: Q001 - Q100

01	Design-pattern foundations	Q001 - Q006
02	SOLID and design judgment	Q007 - Q012
03	Singleton	Q013 - Q015
04	Factory Method	Q016 - Q018
05	Abstract Factory	Q019 - Q021
06	Builder	Q022 - Q024
07	Prototype	Q025 - Q027
08	Adapter	Q028 - Q030
09	Bridge	Q031 - Q033
10	Composite	Q034 - Q036
11	Decorator	Q037 - Q039
12	Facade	Q040 - Q042
13	Flyweight	Q043 - Q045

Bonus section: 12 compiled Java labs, an order-placement design scenario, mock interviews and a 30-day route.

GUIDED TABLE OF CONTENTS / CONTINUED

28 topic chapters

Original interview question sequence retained: Q001 - Q100

15	Chain of Responsibility	Q049 - Q051
16	Command	Q052 - Q054
17	Interpreter	Q055 - Q057
18	Iterator	Q058 - Q060
19	Mediator	Q061 - Q063
20	Memento	Q064 - Q066
21	Observer	Q067 - Q069
22	State	Q070 - Q072
23	Strategy	Q073 - Q075
24	Template Method	Q076 - Q078
25	Visitor	Q079 - Q081
26	Patterns that interviewers compare	Q082 - Q087
27	Patterns in Java and Spring	Q088 - Q093

Bonus section: 12 compiled Java labs, an order-placement design scenario, mock interviews and a 30-day route.

PATTERN SELECTION MAP

Creational + Structural

Choose by design intent, not by the shape of a UML diagram.

Singleton	Restrict construction, not safe mutation
Factory Method	A subclass supplies the creation method
Abstract Factory	Choose mutually compatible product families
Builder	Separate optional setup from valid construction
Prototype	Copy state while controlling aliases
Adapter	Translate different interfaces and outcomes
Bridge	Allow two dimensions to vary independently
Composite	Offer common operation for leaf and tree
Decorator	Keep contract while composing responsibilities
Facade	Simplify a complex use-case entry point
Flyweight	Share intrinsic immutable state
Proxy	Control access through an equivalent contract

PATTERN SELECTION MAP

Behavioral

Choose by design intent, not by the shape of a UML diagram.

Chain of Responsibility	Route until handled or deliberately refused
Command	Represent an operation as a value
Interpreter	Give a small language clear semantics
Iterator	Expose traversal without representation
Mediator	Centralize colleague interaction
Memento	Save opaque immutable state
Observer	Notify subscribers with explicit lifecycle
State	Behavior varies by valid current state
Strategy	Swap algorithms behind shared contract
Template Method	Freeze skeleton and vary limited steps
Visitor	Add operations to a stable type family

DAILY PRACTICE PLAN

30 days to explain, code, and defend

Plan: 25 minutes concept + 20 minutes diagram/code + 15 minutes speaking + 10 minutes failure-case checks. Make the first two weeks foundations, next 10 days implementation, final week senior reasoning.

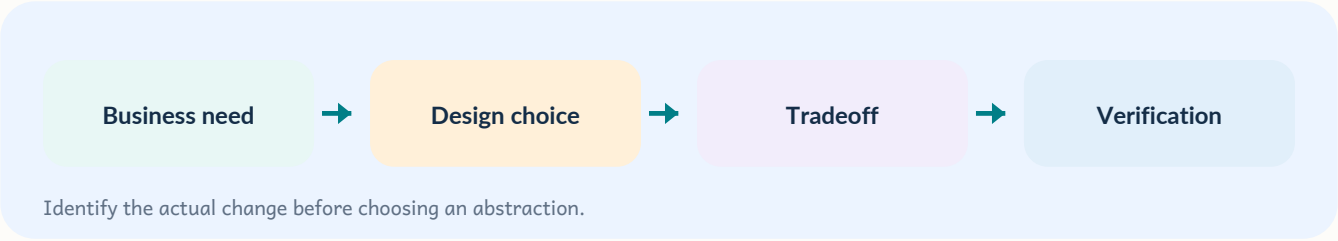
1-3	Intent, SOLID, Singleton
4-7	Factory Method, Abstract Factory, Builder, Prototype
8-11	Adapter, Bridge, Composite, Decorator
12-15	Facade, Flyweight, Proxy, Chain
16-19	Command, Interpreter, Iterator, Mediator
20-23	Memento, Observer, State, Strategy
24-26	Template Method, Visitor, comparisons
27-28	Spring patterns + order/loan scenarios
29	Mock interviews A and B
30	Redo your weakest three Java labs

CHAPTER 01

Design-pattern foundations

Identify the actual change before choosing an abstraction.

ROLE DIAGRAM / THE MAIN COLLABORATION



Identify the actual change before choosing an abstraction.

WHAT TO MASTER

- **Q001** What is a design pattern?
- **Q002** What are the three GoF pattern categories?
- **Q003** How do design patterns differ from architectural patterns?

INTERVIEW TIP

Explain the actual domain constraint first, then name the smallest pattern that preserves it.

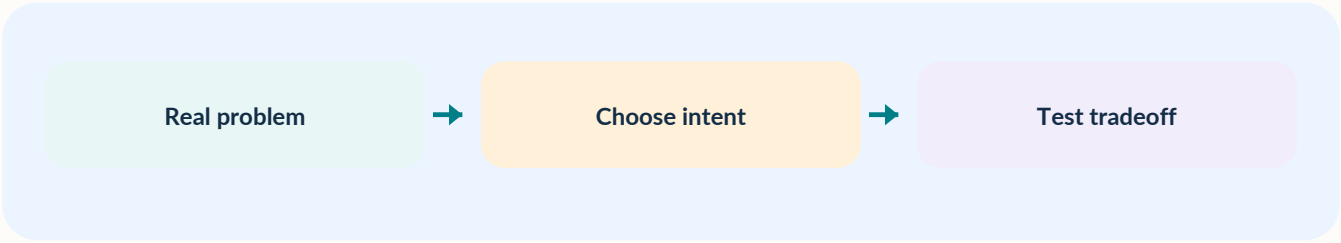
Q001 FOUNDATION

What is a design pattern?

THE SHORT ANSWER

A design pattern describes a reusable collaboration of objects for a recurring design problem in a context.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Problem: several payment algorithms vary independently.
   Design: checkout delegates to a PaymentStrategy.
```

IN PLAIN ENGLISH

A pattern is not a library or a copy-paste class diagram. Adapt its roles to the actual requirements and explain the change it makes easier. The implementation can be small, including a Java function.

INTERVIEW TRAP

More pattern classes do not automatically improve design.

FOLLOW-UP EXPLAINED

What requirement justifies the extra abstraction?
Answer: Add an abstraction when a second real behavior must be substituted, tested separately, or changed independently. If no change axis exists, keep the simpler method.

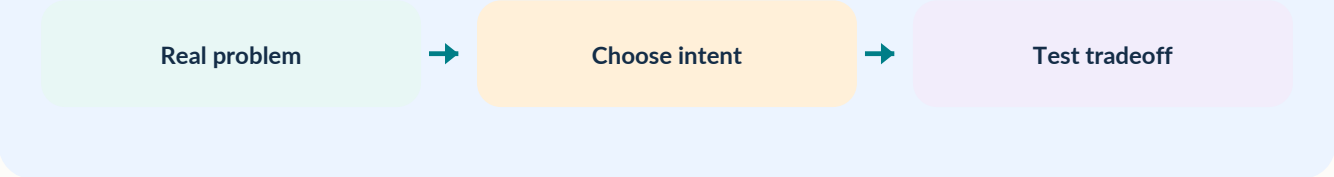
Q002 FOUNDATION

What are the three GoF pattern categories?

THE SHORT ANSWER

Creational patterns organize object creation; structural patterns organize composition; behavioral patterns organize responsibilities and interactions.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Creation: Builder
   Structure: Adapter
   Behavior: Strategy
```

IN PLAIN ENGLISH

The GoF catalog contains 23 patterns. Categories describe intent, not which Java keyword is used. Factory Method uses inheritance, while Abstract Factory commonly uses composition to create a family.

INTERVIEW TRAP

Similar class shapes can express different intents.

FOLLOW-UP EXPLAINED

Why can a wrapper be either Adapter or Decorator?
Answer: A wrapper is an Adapter when it converts one contract to another. It is a Decorator when it preserves the contract but adds behavior such as tracing or compression.

Q003 FOUNDATION

How do design patterns differ from architectural patterns?

THE SHORT ANSWER

Design patterns usually describe object-level collaboration; architectural patterns describe broader system organization and constraints.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Strategy: choose a discount algorithm
   Hexagonal architecture: isolate domain from delivery and persistence
```

IN PLAIN ENGLISH

Layered architecture, event-driven architecture and ports/adapters operate at different scales from GoF patterns. A system may use several together. Explain their boundaries instead of calling every useful arrangement a GoF pattern.

INTERVIEW TRAP

Repository and outbox are useful patterns but are not among the 23 GoF entries.

FOLLOW-UP EXPLAINED

Where would an Adapter sit in hexagonal architecture?
Answer: An Adapter sits on the infrastructure side of a port. Domain code calls the port; the adapter translates that call into a database, HTTP, or vendor API.

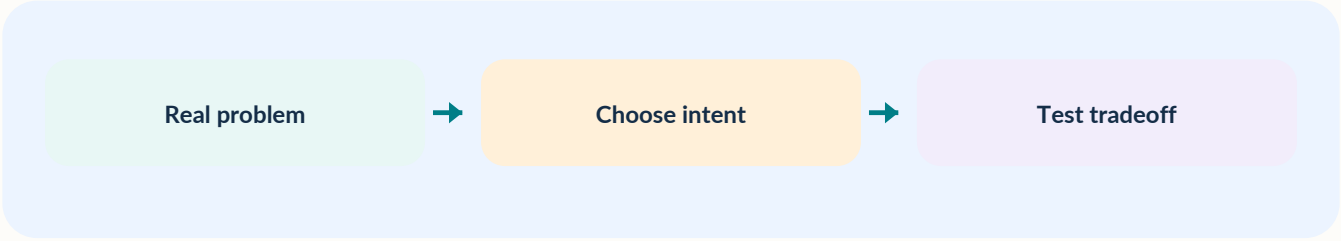
Q004 FOUNDATION

What does favor composition over inheritance mean?

THE SHORT ANSWER

Prefer delegation when behavior should vary without coupling it to one fixed class hierarchy.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
class Checkout {
     private final DiscountPolicy discount;
   }
```

IN PLAIN ENGLISH

Composition can swap independently varying behavior and reduce fragile superclass dependencies. Inheritance remains useful for genuine substitutable abstractions and stable algorithm skeletons. Do not turn the principle into an absolute ban.

INTERVIEW TRAP

Delegation still needs a coherent contract and lifecycle.

FOLLOW-UP EXPLAINED

When is Template Method a reasonable choice?
Answer: Template Method fits a stable sequence with limited, well-defined steps subclasses must implement. Use composition when steps need independent runtime selection.

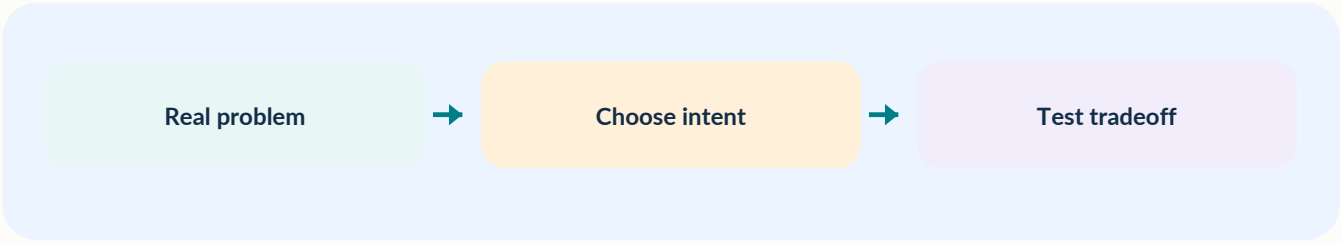
Q005 FOUNDATION

What does program to an interface mean?

THE SHORT ANSWER

Depend on the smallest useful abstraction or contract rather than unnecessary implementation details.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
class OrderService {
     private final PaymentPort payments;
   }
```

IN PLAIN ENGLISH

An interface can be a Java interface or a stable abstraction. It should reflect the consumer's needs and permit meaningful substitution. Creating an interface for every class without variation or boundary value can add noise.

INTERVIEW TRAP

An interface does not automatically eliminate coupling to concrete semantics.

FOLLOW-UP EXPLAINED

What error/outcome contract should PaymentPort expose?
Answer: Return a documented success, rejection, or unknown outcome, with stable IDs and error codes. A timeout must not automatically mean a payment definitely failed.

Q006 FOUNDATION

When should you avoid introducing a pattern?

THE SHORT ANSWER

Avoid it when it solves no current variation, makes the simple path harder, or hides an important invariant.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Two stable branches in one small method
   May be clearer than a registry + factories + reflection
```

IN PLAIN ENGLISH

Use the simplest structure that keeps the relevant change safe. Introduce a pattern when duplication, variation or dependency boundaries provide evidence. Measure readability, testing and change impact rather than pattern count.

INTERVIEW TRAP

Speculative extension points can become maintenance obligations.

FOLLOW-UP EXPLAINED

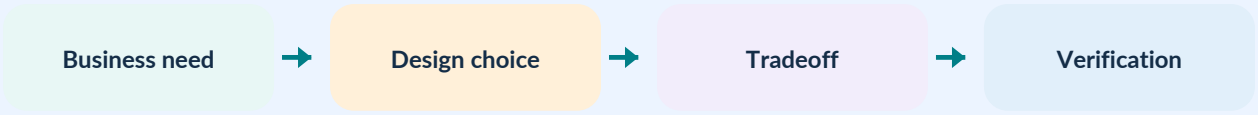
What concrete change would trigger refactoring?
Answer: Refactor when a third distinct algorithm arrives, branches have different owners, or a new dependency needs its own test boundary. Show this with change history.

CHAPTER 02

SOLID and design judgment

Use behavioral contracts and focused responsibilities, not interface counts.

ROLE DIAGRAM / THE MAIN COLLABORATION



Use behavioral contracts and focused responsibilities, not interface counts.

WHAT TO MASTER

- **Q007** What is the Single Responsibility Principle?
- **Q008** What is the Open/Closed Principle?
- **Q009** What is the Liskov Substitution Principle?

INTERVIEW TIP

Explain the actual domain constraint first, then name the smallest pattern that preserves it.

Q007 FOUNDATION

What is the Single Responsibility Principle?

THE SHORT ANSWER

A module should group behavior that changes for the same reason and separate behavior driven by different responsibilities.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Invoice calculation != PDF rendering != payment persistence
```

IN PLAIN ENGLISH

Separate pricing rules from presentation and storage so one policy change need not disturb the others. SRP is not one method per class; related operations can share one coherent responsibility.

INTERVIEW TRAP

A class named Manager can still hide unrelated reasons to change.

FOLLOW-UP EXPLAINED

Who owns each change in this example?
Answer: Pricing owns amounts and discounts, rendering owns PDF layout, and persistence owns database writes. Each responsibility should change for its own business reason.

Q008 FOUNDATION

What is the Open/Closed Principle?

THE SHORT ANSWER

Keep stable policy intact while allowing a required axis of variation through extension.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Checkout uses DiscountPolicy
   Add SeasonalDiscount without editing checkout algorithm
```

IN PLAIN ENGLISH

The composition/configuration point may still change to select the new implementation. OCP does not mean source files are never edited. Preserve a verified stable contract and avoid extensibility for every imagined future need.

INTERVIEW TRAP

Moving a switch into a factory does not remove all change points.

FOLLOW-UP EXPLAINED

Which variation should remain closed?
Answer: Keep the checkout calculation and its invariants stable. Allow discount implementations to vary behind a validated DiscountPolicy contract.

Q009 FOUNDATION

What is the Liskov Substitution Principle?

THE SHORT ANSWER

A replacement implementation must preserve the behavioral contract clients rely on, including valid inputs and promised outcomes.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
ReadableStore contract accepts all supported IDs
   CachedStore must not reject IDs merely because they are uncached
```

IN PLAIN ENGLISH

Avoid stronger preconditions, weaker guarantees and broken invariants. Java type compatibility is not sufficient. Exception and side-effect expectations matter, especially when an adapter fronts a remote service.

INTERVIEW TRAP

A subclass that throws UnsupportedOperationException for promised behavior can violate substitution.

FOLLOW-UP EXPLAINED

How would a contract test detect the violation?
Answer: Run the same contract suite against every ReadableStore implementation, including missing IDs, failures, ordering, and accepted inputs; a subtype must not silently narrow the contract.

Q010 FOUNDATION

What is the Interface Segregation Principle?

THE SHORT ANSWER

Clients should depend on contracts containing the operations they need rather than a broad unrelated interface.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
ReadableStore.find(id)
   WritableStore.save(value)
```

IN PLAIN ENGLISH

Split read-only and mutation capabilities when clients genuinely differ. Keep cohesive methods together instead of fragmenting every operation. This prevents read-only implementations from faking unsupported writes.

INTERVIEW TRAP

Many tiny interfaces can also obscure an understandable contract.

FOLLOW-UP EXPLAINED

Which client benefits from the split?
Answer: Read-only clients, such as reporting screens, depend only on ReadableStore, so they cannot accidentally call writes or implement meaningless save methods.

Q011 FOUNDATION

What is the Dependency Inversion Principle?

THE SHORT ANSWER

High-level policy and low-level implementation should depend on suitable abstractions rather than high-level code depending directly on low-level details.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Domain PaymentPort <- HttpPaymentAdapter
   OrderService depends on PaymentPort
```

IN PLAIN ENGLISH

The business layer can own the port, and infrastructure implements it. Dependency injection supplies an implementation; DIP is about dependency direction and contracts. A container can inject tightly coupled concrete classes without achieving DIP.

INTERVIEW TRAP

DI and DIP are related but not identical.

FOLLOW-UP EXPLAINED

Which package should own the interface?
Answer: Put PaymentPort in the domain or application-core module that needs it. Keep HttpPaymentAdapter in infrastructure so dependencies point inward.

Q012 FOUNDATION

How should SOLID be balanced with simplicity?

THE SHORT ANSWER

Apply it to real sources of change and verify the design remains understandable and efficient.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
One payment provider today, several required next quarter
   A small PaymentPort boundary is justified
```

IN PLAIN ENGLISH

Explain a tradeoff: extra interfaces help replace external infrastructure, while excessive abstractions slow navigation. Patterns should preserve business invariants and make tests meaningful. Prefer explicit contracts over slogan-based redesign.

INTERVIEW TRAP

SOLID is not a score calculated from interface counts.

FOLLOW-UP EXPLAINED

What evidence shows the abstraction paid off?
Answer: Compare change size, number of impacted classes, test isolation, and defect rate before and after. An abstraction earns its cost when real changes become safer.

CHAPTER 03

Singleton

Restrict construction, not safe mutation

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q013** What is Singleton and when is it useful?
- **Q014** How do you implement lazy initialization safely?
- **Q015** Why can Singleton make testing and design harder?

INTERVIEW TIP

Use the Singleton pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q013 CREATIONAL

What is Singleton and when is it useful?

THE SHORT ANSWER

Singleton controls access to one instance within a defined runtime scope.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
enum AppSettings {
     INSTANCE;
     public String region() { return "local"; }
   }
```

IN PLAIN ENGLISH

An enum is a concise Java singleton with well-defined construction/serialization behavior. Define the scope: class-loader identity matters, and multiple processes still have multiple instances. Dependency-injected lifecycle control often avoids static global access.

INTERVIEW TRAP

One instance does not imply safe mutable state.

FOLLOW-UP EXPLAINED

What happens with two class loaders?
Answer: Each class loader may define its own AppSettings class and therefore its own enum singleton instance. It is not one universal object across a JVM.

Q014 CREATIONAL

How do you implement lazy initialization safely?

THE SHORT ANSWER

The initialization-on-demand holder idiom uses Java class initialization to publish a lazy instance safely.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
final class Registry {
     private Registry() {}
     private static class Holder { static final Registry R = new Registry(); }
     static Registry instance() { return Holder.R; }
   }
```

IN PLAIN ENGLISH

Initialization occurs when the holder is first used. For double-checked locking, the instance reference must be volatile and the algorithm must be correct. The holder controls construction but not thread safety of later mutations.

INTERVIEW TRAP

Lazy construction cannot repair unsafe shared maps inside the instance.

FOLLOW-UP EXPLAINED

When would you choose eager creation instead?
Answer: Choose eager creation if the object is cheap, always required, or startup should detect initialization problems immediately; lazy creation is useful for costly optional work.

Q015 CREATIONAL

Why can Singleton make testing and design harder?

THE SHORT ANSWER

Static global access hides dependencies and can leak mutable state between tests or consumers.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Bad: OrderService calls GlobalGateway.instance()
   Better boundary: constructor accepts PaymentPort
```

IN PLAIN ENGLISH

Injecting a scoped instance makes replacement and lifetime explicit. A singleton-scoped stateless service is often practical; a globally accessed mutable singleton is a separate design choice. Tests should isolate unavoidable shared state.

INTERVIEW TRAP

Private constructor alone does not solve reflection, serialization and lifecycle concerns.

FOLLOW-UP EXPLAINED

How would you replace this gateway in a test?
Answer: Inject PaymentPort through the constructor and pass a fake or Mockito double in tests. Avoid tests that edit global static singleton state.

CHAPTER 04

Factory Method

A subclass supplies the creation method

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q016** What is Factory Method?
- **Q017** When is Factory Method worth using?
- **Q018** How can a Factory Method preserve substitutability?

INTERVIEW TIP

Use the Factory Method pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q016 CREATIONAL

What is Factory Method?

THE SHORT ANSWER

A creator exposes an overridable creation method so subclasses can supply the concrete product used by its workflow.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
abstract class Job {
     abstract Parser createParser();
     Result run(String input) { return createParser().parse(input); }
   }
```

IN PLAIN ENGLISH

The workflow depends on Parser while specialized Jobs choose the parser. This is different from any static method named of or create. The extension point should serve a stable workflow rather than add inheritance for one constructor.

INTERVIEW TRAP

A static switch factory is not automatically the GoF Factory Method.

FOLLOW-UP EXPLAINED

Which method does the subclass override?
Answer: The subclass overrides createParser(), not run(). The base run() retains the workflow and calls the chosen Parser.

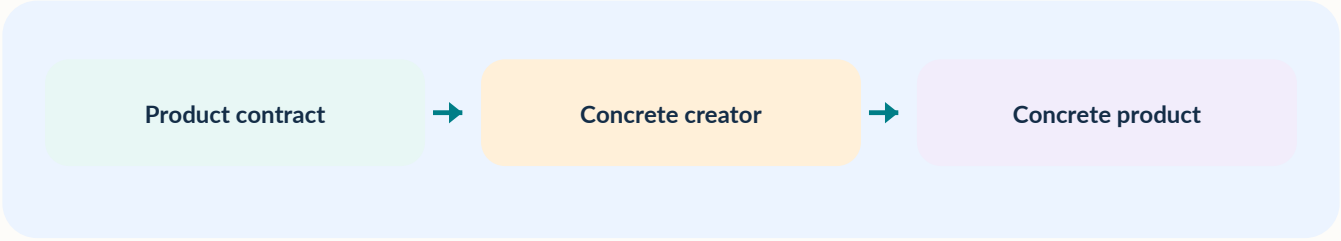
Q017 CREATIONAL

When is Factory Method worth using?

THE SHORT ANSWER

Use it when an existing creator workflow needs a creation variation point aligned with its subclasses.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
CsvImportJob creates CsvParser
   JsonImportJob creates JsonParser
```

IN PLAIN ENGLISH

Each job can retain the shared run algorithm while choosing a product. If parser choice varies independently from job type, injecting a Parser or Supplier may be simpler. Avoid multiplying subclasses for unrelated combinations.

INTERVIEW TRAP

An inheritance hierarchy can become the wrong axis of variation.

FOLLOW-UP EXPLAINED

How would constructor injection change this design?
Answer: Inject Parser into Job's constructor and remove the subclasses if parser choice is the only variation; the shared run algorithm stays in one class.

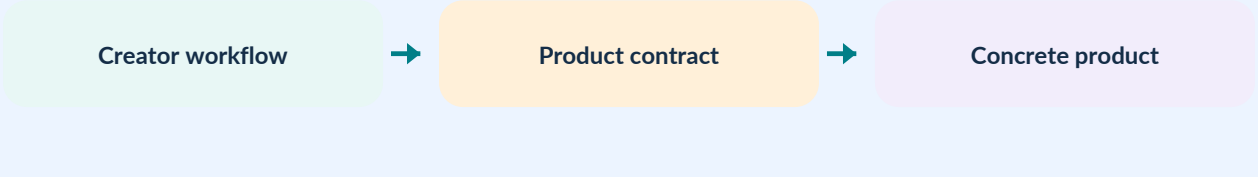
Q018 CREATIONAL

How can a Factory Method preserve substitutability?

THE SHORT ANSWER

Every returned product must satisfy the product contract the creator's workflow uses.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Parser.parse -> Result or documented ParseFailure
   Both CsvParser and JsonParser preserve this contract
```

IN PLAIN ENGLISH

The caller should not require downcasts to concrete parsers. Specify ownership and reuse: a factory may create a new object or provide a suitable existing one. Tests should exercise the shared workflow for every product.

INTERVIEW TRAP

Returning a cached non-thread-safe parser changes concurrency assumptions.

FOLLOW-UP EXPLAINED

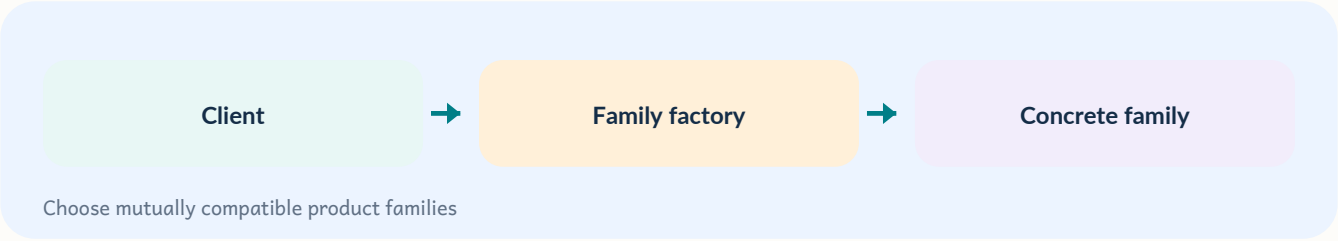
Which product contract would you test?
Answer: Test parsing success, malformed input, null policy, exception shape, side effects, and thread-safety expectations using the same cases for every Parser.

CHAPTER 05

Abstract Factory

Choose mutually compatible product families

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q019 What is Abstract Factory?

Q020 What is the main extension tradeoff?

Q021 How would you use Abstract Factory in backend code?

INTERVIEW TIP

Use the Abstract Factory pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

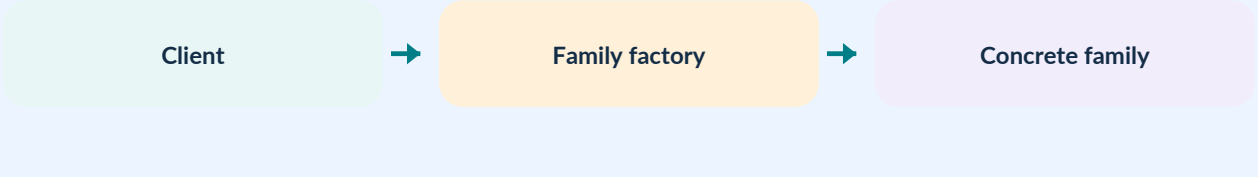
Q019 CREATIONAL

What is Abstract Factory?

THE SHORT ANSWER

It supplies related products through one family-level creation contract while hiding concrete family classes.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface UiFactory {
     Button button();
     Menu menu();
   }
```

IN PLAIN ENGLISH

A DesktopFactory and TouchFactory can provide compatible button/menu pairs. The client chooses a family once and uses its abstractions. Compatibility constraints, not the word Factory in the name, justify the pattern.

INTERVIEW TRAP

Independently mixing family products may violate assumptions.

FOLLOW-UP EXPLAINED

What makes two products belong to one family?
Answer: Products form a family if they must share compatible versions, configuration, style, credentials, or protocols and should be selected together.

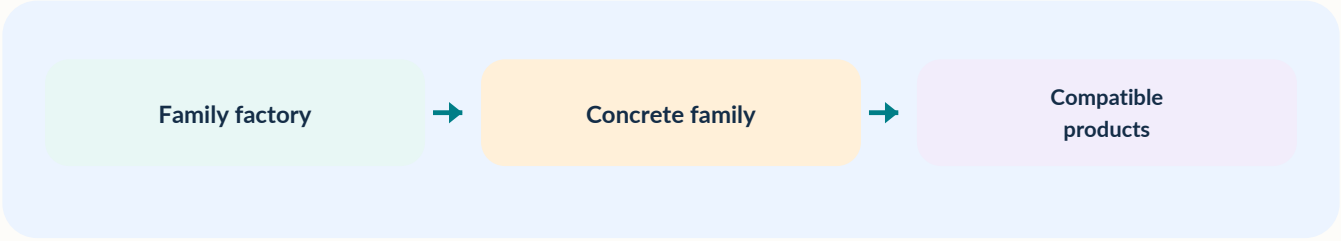
Q020 CREATIONAL

What is the main extension tradeoff?

THE SHORT ANSWER

Adding a family is easy, while adding a new product kind can require changing the abstract factory and every family implementation.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
New family: HighContrastFactory
   New product kind: Tooltip -> update all UiFactory implementations
```

IN PLAIN ENGLISH

This tradeoff differs from Factory Method's single-product variation. If product kinds grow constantly, consider a simpler composition model or a carefully designed registry. Avoid weakening types merely to avoid every interface change.

INTERVIEW TRAP

A string-to-Object factory sacrifices useful compile-time checks.

FOLLOW-UP EXPLAINED

Which dimension grows more often here?
Answer: If you add more deployment families often, Abstract Factory helps. If you continually add product types, every family must be updated; consider simpler ports.

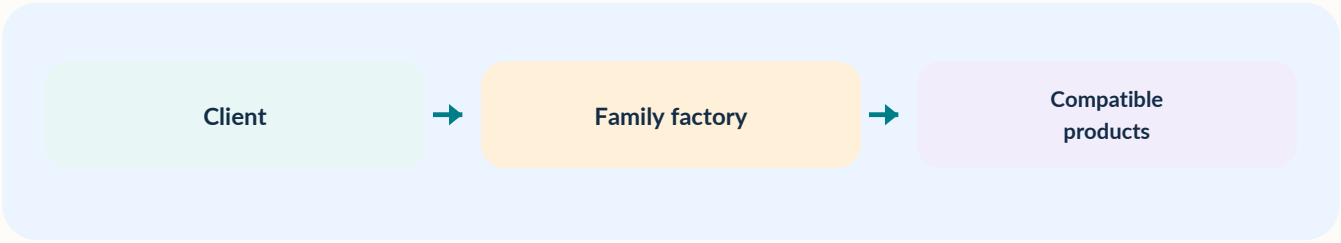
Q021 CREATIONAL

How would you use Abstract Factory in backend code?

THE SHORT ANSWER

Group mutually compatible infrastructure products when a real compatibility requirement exists.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
StorageFamily -> BlobWriter + SignedUrlProvider
   CloudAFamily keeps endpoint and credential policy aligned
```

IN PLAIN ENGLISH

A Spring configuration can assemble the family through DI, but a collection of unrelated bean methods is not proof of Abstract Factory intent. Separate configuration and secrets from business policy. Test family compatibility and failure outcomes.

INTERVIEW TRAP

A family contract should not expose vendor SDK types unnecessarily.

FOLLOW-UP EXPLAINED

When are two separately injected ports sufficient?
Answer: Two injected ports are sufficient when writer and URL signer do not need a coordinated family choice or cross-product compatibility guarantee.

CHAPTER 06

Builder

Separate optional setup from valid construction

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q022 What is Builder?

Q023 How do you make a built object immutable?

Q024 When should you avoid a builder?

INTERVIEW TIP

Use the Builder pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q022 CREATIONAL

What is Builder?

THE SHORT ANSWER

Builder separates stepwise construction from the final representation so complex objects can be assembled clearly.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Request r = new Request.Builder("/orders")
       .timeout(Duration.ofSeconds(2)).header("Accept", "json").build();
```

IN PLAIN ENGLISH

Fluent Java builders often replace telescoping constructors; the GoF form can also use a director to control steps for different representations. Clearly distinguish required fields from optional settings and reject invalid combinations.

INTERVIEW TRAP

A fluent setter chain alone does not guarantee a valid object.

FOLLOW-UP EXPLAINED

Where should cross-field validation run?
Answer: Run validation in build() or the constructed type's constructor, after all fields are available. Reject inconsistent combinations before returning an object.

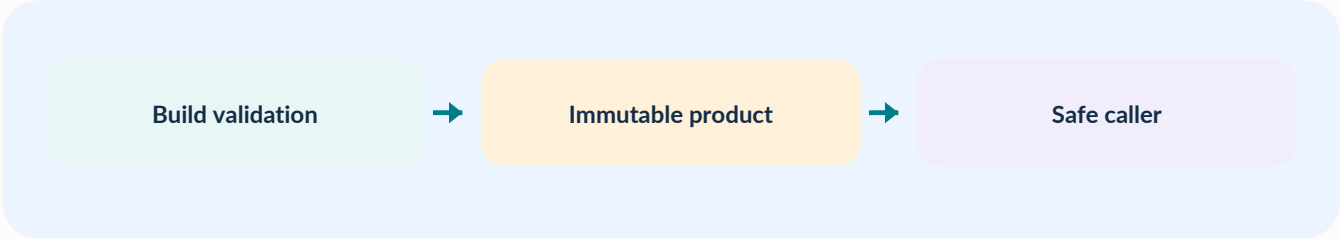
Q023 CREATIONAL

How do you make a built object immutable?

THE SHORT ANSWER

Validate and copy mutable inputs into the result rather than sharing the builder's internal containers.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
this.headers = Map.copyOf(builder.headers);
```

IN PLAIN ENGLISH

Final fields prevent reference reassignment, but not mutation of referenced objects. Deep immutability needs immutable elements or defensive copies at the necessary depth. Decide whether builders are reusable; they are usually not shared across threads.

INTERVIEW TRAP

Unmodifiable view can still reflect changes in its backing collection.

FOLLOW-UP EXPLAINED

What if a header value is itself mutable?
Answer: Copy that value too, or use an immutable representation. Map.copyOf only copies the map structure; it does not deep-copy mutable values.

Q024 CREATIONAL

When should you avoid a builder?

THE SHORT ANSWER

Avoid it for a tiny value whose constructor or record clearly expresses all valid required inputs.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
record Coordinates(double latitude, double longitude) {}
```

IN PLAIN ENGLISH

A builder improves readability when options grow or construction involves constraints. It costs code and can obscure which fields are mandatory. Staged builders can enforce construction steps but increase generic/interface complexity.

INTERVIEW TRAP

A builder for every two-field DTO is often needless ceremony.

FOLLOW-UP EXPLAINED

Would a named static factory communicate intent better?
Answer: Yes. For a small object, a method such as Coordinates.ofDegrees(lat, lon) can validate and explain meaning better than a builder.

CHAPTER 07

Prototype

Copy state while controlling aliases

ROLE DIAGRAM / THE MAIN COLLABORATION



Copy state while controlling aliases

WHAT TO MASTER

Q025 What is Prototype?

Q026 How do shallow and deep copying differ?

Q027 Why is Object.clone not always a good domain API?

INTERVIEW TIP

Use the Prototype pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q025 CREATIONAL

What is Prototype?

THE SHORT ANSWER

Prototype creates a new object by copying a configured example rather than reconstructing it through the usual creation path.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Document draft = template.copy();
   draft.setTitle("Interview Notes");
```

IN PLAIN ENGLISH

Use it when copying a configured graph is the useful operation. Java clone is one mechanism, not the pattern definition. Copy constructors or explicit copy methods often make ownership and validation clearer.

INTERVIEW TRAP

Copying an identifier can accidentally duplicate business identity.

FOLLOW-UP EXPLAINED

Which fields must change in the copied object?
Answer: Generate a fresh business ID, reset timestamps/status where needed, and avoid copying secrets or active handles. Preserve only template configuration intended for reuse.

Q026 CREATIONAL

How do shallow and deep copying differ?

THE SHORT ANSWER

A shallow copy duplicates the outer object but shares nested references; a deep copy duplicates the required mutable graph.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Original and shallowCopy share the same mutable List<Section>
   Deep policy copies sections and their mutable state as needed
```

IN PLAIN ENGLISH

Copy only the state that must be independent. Cycles, shared subgraphs, resources and immutable objects require explicit policy. Blind recursive duplication can lose intended sharing or fail on cycles.

INTERVIEW TRAP

A new ArrayList containing the same mutable elements is not a full deep copy.

FOLLOW-UP EXPLAINED

How would you test aliasing?
Answer: Copy, then mutate the original nested object; the deep copy must remain unchanged. Also mutate the copy to prove independence in both directions.

Q027 CREATIONAL

Why is Object.clone not always a good domain API?

THE SHORT ANSWER

Its default behavior is field copying and its Cloneable contract does not express domain validation or resource ownership.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Document(Document source) {
     this.sections = source.sections.stream().map(Section::copy).toList();
   }
```

IN PLAIN ENGLISH

Explicit copying can reset IDs, exclude secrets and validate invariants. Never duplicate active sockets, database sessions or locks as ordinary values. If cloning is used, document its depth and subclass behavior.

INTERVIEW TRAP

Cloneable does not itself declare a public clone method.

FOLLOW-UP EXPLAINED

How do you copy an object holding a resource handle?
Answer: Do not copy live sockets or database sessions. Reopen or inject a separate handle in the new object and define which object owns closing it.

CHAPTER 08

Adapter

Translate different interfaces and outcomes

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q028** What is Adapter?
- **Q029** How do object and class adapters differ?
- **Q030** How do you test an adapter well?

INTERVIEW TIP

Use the Adapter pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q028 STRUCTURAL

What is Adapter?

THE SHORT ANSWER

Adapter presents the interface a client needs while translating calls to a different existing interface.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
final class PaymentAdapter implements PaymentPort {
     private final VendorClient client;
     public Receipt pay(Money m) { return map(client.charge(toRequest(m))); }
   }
```

IN PLAIN ENGLISH

The adapter isolates vendor request/response types and error mapping. Its purpose is compatibility rather than simply adding features. Contract conversion includes units, identity and failure semantics, not only method names.

INTERVIEW TRAP

Mapping vendor timeout to definite failure can trigger unsafe repeated charges.

FOLLOW-UP EXPLAINED

Which outcomes are uncertain?
Answer: Vendor timeout or connection loss after submission may leave the payment outcome unknown. Query by stable idempotency ID before retrying the charge.

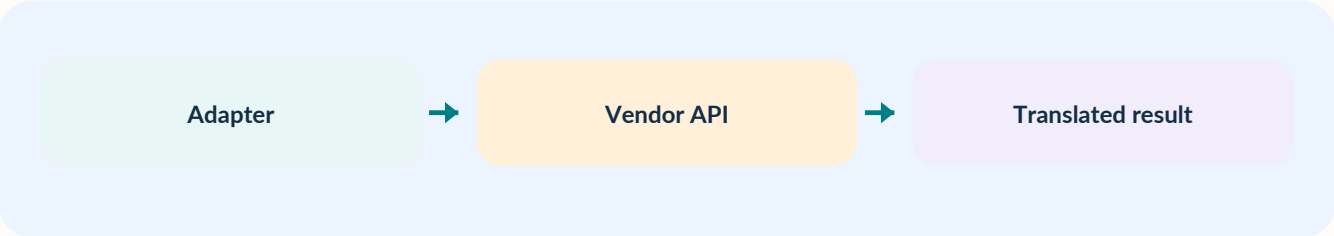
Q029 STRUCTURAL

How do object and class adapters differ?

THE SHORT ANSWER

Object adapters delegate to a wrapped instance; class adapters use inheritance to adapt behavior.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Object adapter: has VendorClient
   Class adapter: extends a suitable adaptee and implements target
```

IN PLAIN ENGLISH

Java's single class inheritance makes object composition broadly useful. Composition lets the adapter wrap supplied implementations and avoids inheriting unrelated behavior. A class adapter can fit a constrained inheritance situation.

INTERVIEW TRAP

Java cannot extend two concrete classes to copy a multiple-inheritance diagram.

FOLLOW-UP EXPLAINED

What dependency would you inject for testing?
Answer: Inject a small VendorClient interface or fake adaptee into the object adapter. Assert translated requests, errors, and responses without real network calls.

Q030 STRUCTURAL

How do you test an adapter well?

THE SHORT ANSWER

Use target-contract tests plus translation tests for successful, rejected and ambiguous upstream outcomes.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Vendor amount in paise -> Money in agreed currency
   Vendor timeout -> documented unknown-payment outcome
```

IN PLAIN ENGLISH

Mock the vendor boundary for precise translation cases, then add integration evidence for serialization/protocol behavior. The application should depend on PaymentPort rather than downcast to access the client.

INTERVIEW TRAP

Matching signatures does not prove semantic compatibility.

FOLLOW-UP EXPLAINED

How do you preserve vendor error context safely?
Answer: Capture only safe vendor error category and correlation ID; redact secrets, payment data, and tokens. Map raw vendor failures to a stable domain error.

CHAPTER 09

Bridge

Allow two dimensions to vary independently

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q031 What is Bridge?

Q032 How does Bridge avoid a class explosion?

Q033 How is Bridge different from Adapter?

INTERVIEW TIP

Use the Bridge pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q031 STRUCTURAL

What is Bridge?

THE SHORT ANSWER

Bridge separates an abstraction from its implementation hierarchy so both can evolve independently.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
abstract class Notification {
     protected final Channel channel;
   }
   // Alert/Receipt notifications x Email/Sms channels
```

IN PLAIN ENGLISH

The notification abstraction delegates delivery to Channel implementations. New notification behavior and new delivery channels need not create a subclass for every combination. Both sides still need a coherent contract.

INTERVIEW TRAP

Ordinary composition alone is not enough to establish Bridge intent.

FOLLOW-UP EXPLAINED

Which two dimensions vary independently?
Answer: Notification type (Alert/Receipt) and transport (Email/SMS) vary independently. Compose one of each rather than creating every combined subclass.

Q032 STRUCTURAL

How does Bridge avoid a class explosion?

THE SHORT ANSWER

It composes two varying dimensions instead of encoding their Cartesian product in one inheritance hierarchy.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Without bridge: AlertEmail, AlertSms, ReceiptEmail, ReceiptSms
   With bridge: Alert, Receipt + EmailChannel, SmsChannel
```

IN PLAIN ENGLISH

This saves duplication when the dimensions are real. It does not mean every combination is valid; validate constraints at assembly. If only one small dimension varies, a Strategy may be the simpler explanation.

INTERVIEW TRAP

Allowing invalid combinations is not useful flexibility.

FOLLOW-UP EXPLAINED

Where should channel compatibility be checked?
Answer: Check supported combinations at the assembly/configuration boundary, where both the notification type and Channel implementation are selected.

Q033 STRUCTURAL

How is Bridge different from Adapter?

THE SHORT ANSWER

Bridge is designed to separate variation dimensions; Adapter makes an existing interface usable through a different expected interface.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Bridge: choose notification kind and channel independently
   Adapter: wrap a vendor SMS client as Channel
```

IN PLAIN ENGLISH

The patterns can coexist: a Channel implementation may itself be an adapter. Describe intent and timing of the design rather than judging solely from a reference field and delegation method.

INTERVIEW TRAP

Both may wrap an object, but solve different problems.

FOLLOW-UP EXPLAINED

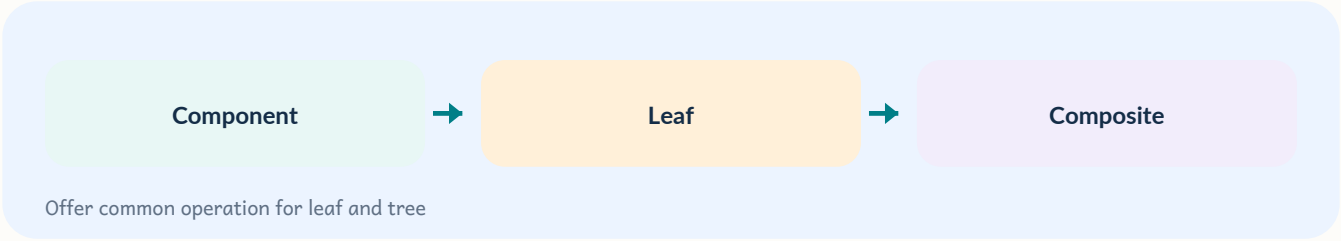
Can you identify both boundaries in one diagram?
Answer: Bridge separates Notification from Channel choice. An Adapter may sit beneath Channel to translate one vendor SMS SDK into the Channel contract.

CHAPTER 10

Composite

Offer common operation for leaf and tree

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q034** What is Composite?
- **Q035** Should leaf objects expose add/remove operations?
- **Q036** How do you handle performance and ownership in Composite?

INTERVIEW TIP

Use the Composite pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

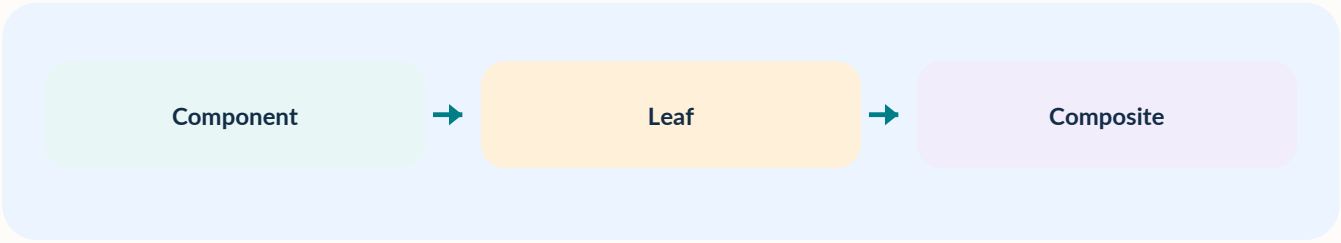
Q034 STRUCTURAL

What is Composite?

THE SHORT ANSWER

Composite represents a part-whole tree where leaves and groups support a common operation.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface CostItem { int cents(); }
   // Product is a leaf; Bundle sums child CostItems.
```

IN PLAIN ENGLISH

Clients can price a Product or Bundle through CostItem. Use a tree or explicitly define shared-node graph semantics. A group recursively delegates the operation to its children.

INTERVIEW TRAP

Recursion is unsafe if cycles are allowed accidentally.

FOLLOW-UP EXPLAINED

How would you prevent a bundle containing itself?
Answer: Reject insertion if a child is this bundle or already reaches it through descendants. An immutable tree and visited-node checks can enforce acyclicity.

Q035 STRUCTURAL

Should leaf objects expose add/remove operations?

THE SHORT ANSWER

Choose between a uniform but partly unsupported interface and a safer split where only composites expose child management.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
CostItem -> cents()
   Bundle -> add(CostItem), remove(CostItem)
```

IN PLAIN ENGLISH

Keeping add/remove out of leaves avoids unsupported operations and can better preserve substitution. Uniform management APIs can simplify some clients but need a deliberate contract. Immutable trees are another useful choice.

INTERVIEW TRAP

Returning fake success from leaf.add breaks expectations.

FOLLOW-UP EXPLAINED

Which clients actually need tree editing?
Answer: Only editors, builders, and admin tools that change bundle structure need add/remove operations; ordinary pricing clients need cents() only.

Q036 STRUCTURAL

How do you handle performance and ownership in Composite?

THE SHORT ANSWER

Bound depth, define child ownership and cache derived totals only with a valid invalidation policy.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Bundle(List<CostItem> children) uses List.copyOf(children)
```

IN PLAIN ENGLISH

Immutable children make aggregate computation easier to reason about. Large/deep trees may require iterative traversal or incremental summaries. Shared mutable children can change several parents unexpectedly.

INTERVIEW TRAP

A cached total becomes stale if a child changes without notification.

FOLLOW-UP EXPLAINED

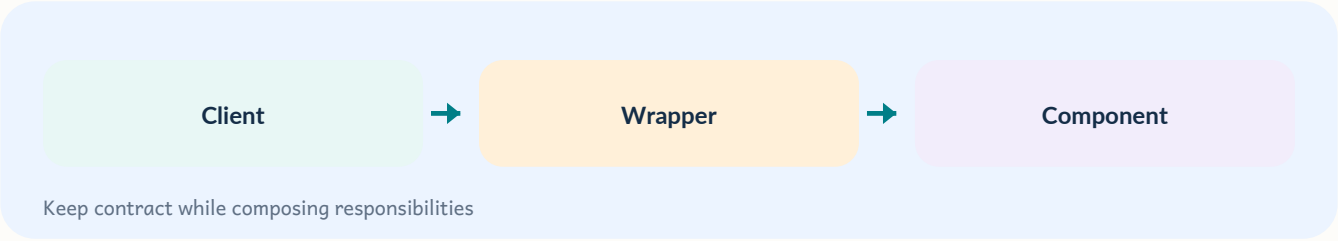
How do you test a shared child update?
Answer: Update one shared child and assert each parent reflects the agreed ownership model; if parents are immutable, confirm shared mutable children are rejected.

CHAPTER 11

Decorator

Keep contract while composing responsibilities

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q037** What is Decorator?
- **Q038** Why does decorator order matter?
- **Q039** What are common Decorator costs?

INTERVIEW TIP

Use the Decorator pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

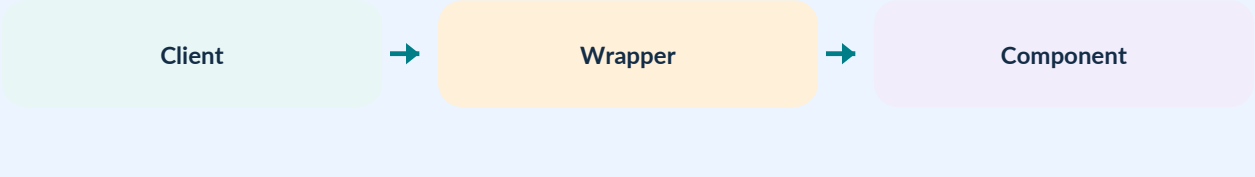
Q037 STRUCTURAL

What is Decorator?

THE SHORT ANSWER

Decorator adds behavior by wrapping an object behind the same component contract.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface Sender { void send(Message m); }
   // LoggingSender wraps Sender; CompressionSender wraps Sender.
```

IN PLAIN ENGLISH

Decorators can be stacked at assembly time without multiplying subclasses for every feature combination. The wrapper must preserve required semantics and deliberately choose behavior before/after delegation.

INTERVIEW TRAP

A wrapper that changes the expected interface is more likely an Adapter.

FOLLOW-UP EXPLAINED

Which order should the wrappers use?
Answer: Choose order from the required behavior: authorize before send, log appropriate data, then compress before encryption. Test the actual nested call path.

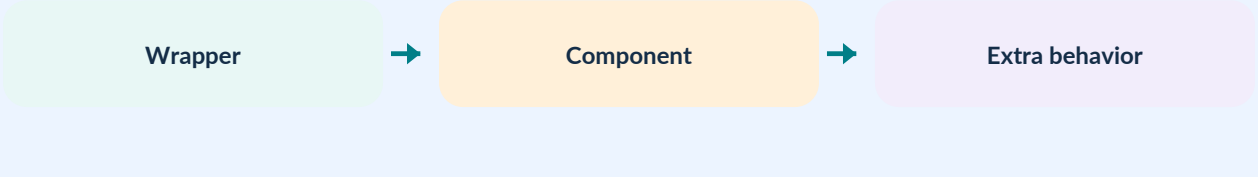
Q038 STRUCTURAL

Why does decorator order matter?

THE SHORT ANSWER

Each wrapper sees the call and data at its position, so changing nesting can change effects, timing and errors.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Retry(Tracing(Sender)) -> each retry can get a trace
   Tracing(Retry(Sender)) -> one outer logical operation span
```

IN PLAIN ENGLISH

Choose the intended observability, authorization and failure boundary. Compression then encryption differs from encryption then compression. Tests should assert operation order and failure behavior, not only final success.

INTERVIEW TRAP

Retry wrappers can duplicate nonidempotent effects.

FOLLOW-UP EXPLAINED

What should happen when logging itself fails?
Answer: A noncritical logging failure should usually be isolated and reported without changing successful delivery. Never silently ignore an audit failure when audit is mandatory.

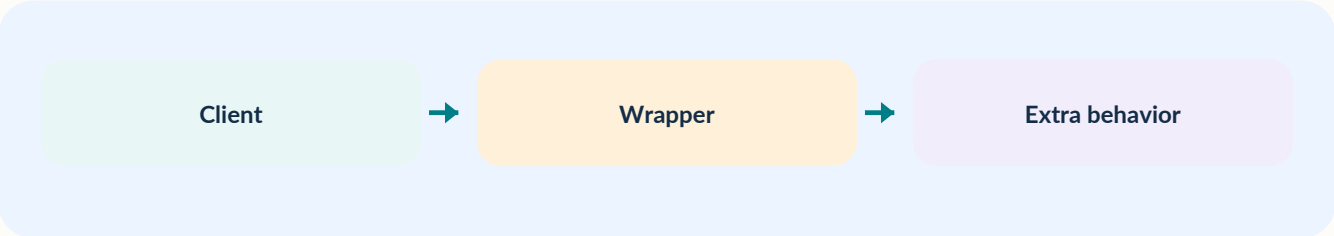
Q039 STRUCTURAL

What are common Decorator costs?

THE SHORT ANSWER

Additional wrappers complicate identity, debugging and lifecycle while potentially hiding semantic changes.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
wrappedSender != originalSender by reference
```

IN PLAIN ENGLISH

Avoid downcasts or identity checks that assume the wrapper is the concrete component. Define resource closing and ownership to prevent double-close or leaked resources. Prefer explicit assembly for a small known stack.

INTERVIEW TRAP

Same interface does not justify violating the original contract.

FOLLOW-UP EXPLAINED

Who closes the wrapped connection?
Answer: The owner that creates the wrapping stack must define shutdown. Close once, usually from the outermost owner, with decorators following an agreed contract.

CHAPTER 12

Facade

Simplify a complex use-case entry point

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q040 What is Facade?

Q041 How is Facade different from Mediator?

Q042 How do you prevent a facade becoming a god object?

INTERVIEW TIP

Use the Facade pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q040 STRUCTURAL

What is Facade?

THE SHORT ANSWER

Facade supplies a simpler cohesive interface over a set of subsystem operations.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
CheckoutFacade.placeOrder(request)
   // Coordinates pricing, stock and payment collaborators.
```

IN PLAIN ENGLISH

The client need not know every subsystem step. A facade should delegate rather than absorb all domain policy into a giant service. Its purpose is usability and coupling reduction.

INTERVIEW TRAP

One method call does not mean one atomic distributed operation.

FOLLOW-UP EXPLAINED

What happens if payment succeeds and shipment fails?
Answer: Do not call the whole operation atomic. Track order state and use a compensating or recovery workflow to retry shipment or refund when business policy requires.

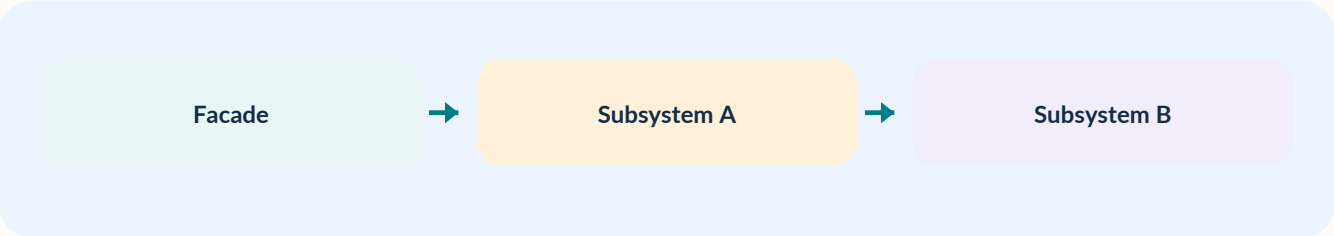
Q041 STRUCTURAL

How is Facade different from Mediator?

THE SHORT ANSWER

Facade simplifies external access; Mediator organizes interaction among peer collaborators.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Facade: caller uses one checkout API
   Mediator: form controls communicate through a coordinator
```

IN PLAIN ENGLISH

A facade's subsystems need not know it exists. Mediated colleagues intentionally route coordination through the mediator. One component can play both roles, but explaining separate responsibilities keeps the design clear.

INTERVIEW TRAP

Every orchestration class is not automatically both patterns.

FOLLOW-UP EXPLAINED

Which objects know the coordinator?
Answer: The client knows the Facade, but subsystems need not. In Mediator, collaborating controls know or communicate through the coordinator.

Q042 STRUCTURAL

How do you prevent a facade becoming a god object?

THE SHORT ANSWER

Keep the public use case coherent and move independent domain rules and infrastructure translation to collaborators.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Facade coordinates PaymentPort, PricingPolicy and OrderRepository
```

IN PLAIN ENGLISH

Test the use-case outcome and failure sequence. Separate commands and reads when they differ materially. If several unrelated teams must edit the facade for every change, its responsibility is likely too broad.

INTERVIEW TRAP

Delegation can still become tangled if ordering rules are implicit.

FOLLOW-UP EXPLAINED

Which responsibility would you extract first?
Answer: Move the most independent business policy, such as pricing rules, into its own tested collaborator and keep the facade as use-case orchestration.

CHAPTER 13

Flyweight

Share intrinsic immutable state

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q043 What is Flyweight?

Q044 How is Flyweight different from a normal cache?

Q045 What can go wrong with a Flyweight pool?

INTERVIEW TIP

Use the Flyweight pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

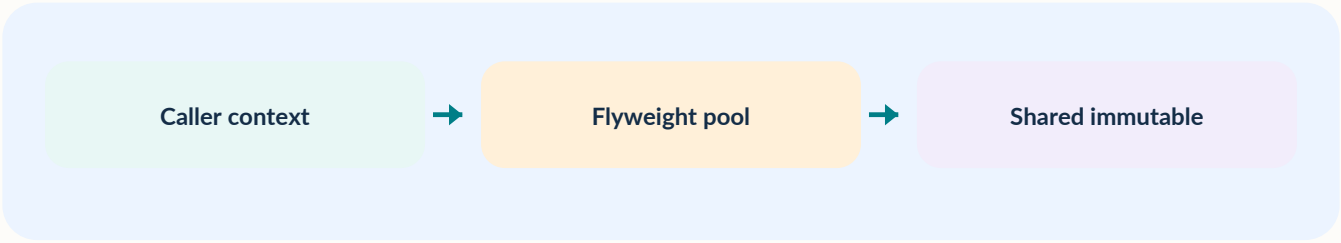
Q043 STRUCTURAL

What is Flyweight?

THE SHORT ANSWER

Flyweight shares common intrinsic state across many logical objects while callers supply differing extrinsic context.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
GlyphStyle(font, size, color) shared
   Glyph position supplied when drawing
```

IN PLAIN ENGLISH

It reduces duplication when shared data is substantial and many instances repeat it. Separate intrinsic state from per-use position, owner or request identity. The shared value should normally be immutable.

INTERVIEW TRAP

A shared object containing mutable request state creates cross-call leakage.

FOLLOW-UP EXPLAINED

Which fields are safe to share?
Answer: Share immutable fonts, styles, static metadata, and configuration. Keep position, request ID, tenant context, and mutable state outside the flyweight.

Q044 STRUCTURAL

How is Flyweight different from a normal cache?

THE SHORT ANSWER

Flyweight models shared identity/state for many logical instances; a cache broadly retains previously computed or loaded results.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
StylePool returns one immutable style for equivalent StyleKey
   Product cache stores a source result with expiry
```

IN PLAIN ENGLISH

A flyweight pool may be implemented with a map/cache, but the design intent is state factoring. Lifetime and eviction depend on equality/identity assumptions and actual memory savings. Measure retained memory.

INTERVIEW TRAP

A map named pool is not automatically a Flyweight.

FOLLOW-UP EXPLAINED

Does the client require reference identity?
Answer: Not necessarily. If clients only care about equivalent immutable values, equality is enough; reference identity should be part of the contract only when required.

Q045 STRUCTURAL

What can go wrong with a Flyweight pool?

THE SHORT ANSWER

Unbounded unique keys can consume more memory than sharing saves, and mutable keys/state can corrupt reuse.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
ConcurrentHashMap<StyleKey, GlyphStyle> with stable value keys
```

IN PLAIN ENGLISH

Use bounded domains or reviewed retention policies. computeIfAbsent can coordinate construction within the map's contract, but loaders should avoid inappropriate recursion/blocking. Correct equality and hash codes define which styles are shared.

INTERVIEW TRAP

Including unique requestId in StyleKey defeats sharing.

FOLLOW-UP EXPLAINED

How would you prove memory savings?
Answer: Benchmark allocated objects and retained heap for realistic cardinality, plus hit ratio and pool overhead. An unbounded map may cost more than sharing saves.

CHAPTER 14

Proxy

Control access through an equivalent contract

ROLE DIAGRAM / THE MAIN COLLABORATION



Control access through an equivalent contract

WHAT TO MASTER

Q046 What is Proxy?

Q047 How do JDK dynamic proxies and subclass proxies differ?

Q048 Why does a self-call bypass common Spring proxy advice?

INTERVIEW TIP

Use the Proxy pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

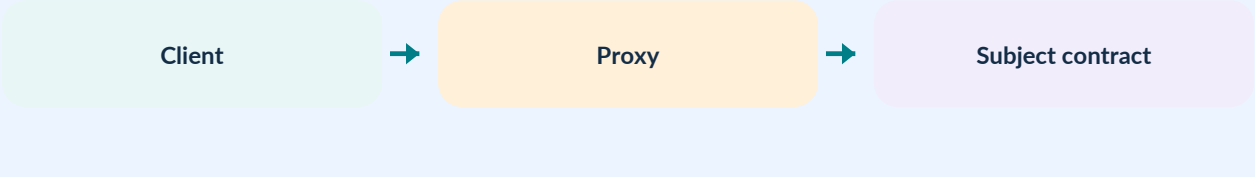
Q046 STRUCTURAL

What is Proxy?

THE SHORT ANSWER

Proxy stands in for a subject and controls access, such as authorization, lazy creation or remote invocation.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface Report { Data load(); }
   // AuthorizedReportProxy checks permission then delegates.
```

IN PLAIN ENGLISH

The client uses the subject contract while the proxy manages an access concern. This differs from Decorator's primary intent of composable responsibility addition, although structures can look similar. Preserve documented exceptions and results.

INTERVIEW TRAP

A remote proxy cannot make a network call behave like a guaranteed local call.

FOLLOW-UP EXPLAINED

What timeout semantics should it expose?
Answer: Expose deadlines and timeout exceptions clearly. A timeout says the remote result may be unknown, not that the action was never performed.

Q047 STRUCTURAL

How do JDK dynamic proxies and subclass proxies differ?

THE SHORT ANSWER

JDK dynamic proxies implement interfaces; subclass proxies derive from a concrete class and inherit its overridability constraints.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Proxy.newProxyInstance(loader, interfaces, invocationHandler)
```

IN PLAIN ENGLISH

JDK proxy invocation routes through an InvocationHandler. A subclass cannot override final methods; private methods are not normal override join points. Framework defaults depend on configuration. Handle Object methods and exception translation appropriately.

INTERVIEW TRAP

Never assume every Spring bean uses the same proxy type.

FOLLOW-UP EXPLAINED

Why can a final method escape advice?
Answer: Subclass proxies cannot override final methods, so proxy interception cannot apply advice to that call. JDK dynamic proxies advise interface methods through their proxy.

Q048 STRUCTURAL

Why does a self-call bypass common Spring proxy advice?

THE SHORT ANSWER

An internal call executes on the target without passing through the external proxy interception boundary.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
caller -> proxy -> target.outer()
   target.outer() -> this.inner() -> no external proxy crossing
```

IN PLAIN ENGLISH

This affects default AOP, cache and transactional arrangements. Extract the advised operation into a separate injected service when needed. Aspect weaving has different mechanics, so identify the actual mode.

INTERVIEW TRAP

Adding an annotation does not prove the call is intercepted.

FOLLOW-UP EXPLAINED

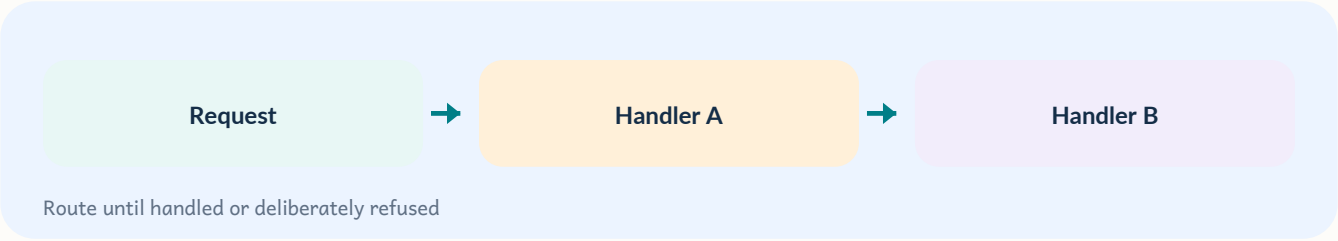
How would you test the real bean's advice path?
Answer: Call the Spring-managed bean from another bean and assert a real transactional/cache/async effect. Compare with a self-call to demonstrate bypass.

CHAPTER 15

Chain of Responsibility

Route until handled or deliberately refused

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q049** What is Chain of Responsibility?
- **Q050** How do you design a safe chain contract?
- **Q051** How does a chain differ from a fixed pipeline?

INTERVIEW TIP

Use the Chain of Responsibility pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

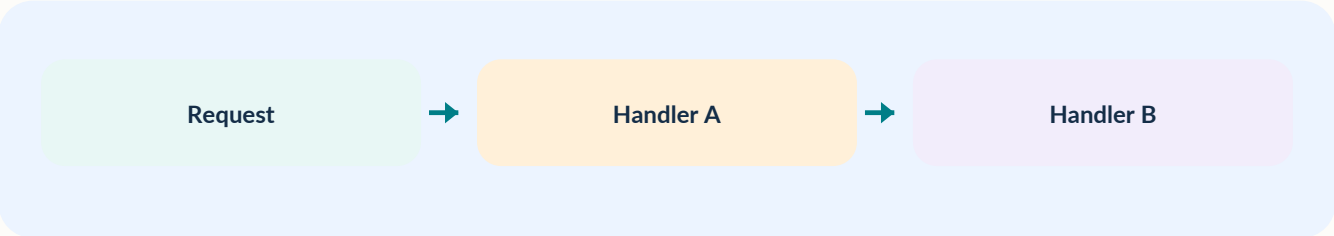
Q049 BEHAVIORAL

What is Chain of Responsibility?

THE SHORT ANSWER

It gives multiple handlers a chance to process a request, reducing sender coupling to a specific receiver.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Request -> authentication handler -> quota handler -> endpoint
```

IN PLAIN ENGLISH

Classical chains may stop at a suitable handler; validation/filter variants can execute multiple steps before continuing. Specify what handled, rejected and passed-on mean. Order must be visible and testable.

INTERVIEW TRAP

A request should not silently vanish at the end.

FOLLOW-UP EXPLAINED

What is the default terminal behavior?
Answer: Specify fail-closed terminal behavior, such as reject or explicit unhandled result. Never silently authorize a request when no handler accepts it.

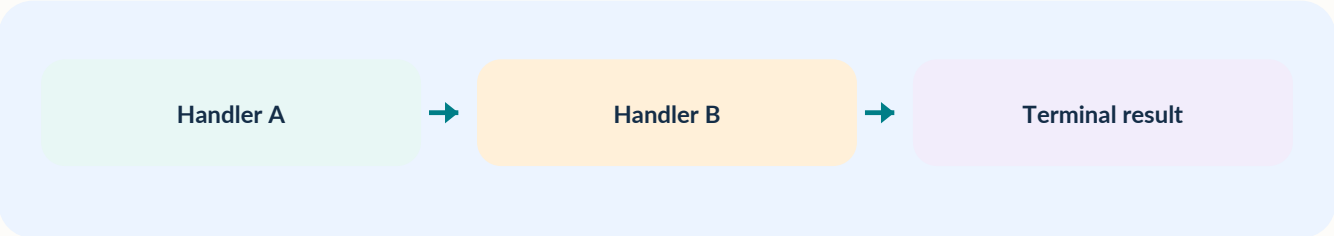
Q050 BEHAVIORAL

How do you design a safe chain contract?

THE SHORT ANSWER

Represent continuation and rejection explicitly, and define who owns progression through the chain.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Decision = Continue | Reject(reason) | Handled(result)
```

IN PLAIN ENGLISH

A linked handler or coordinator-owned list can work. Prevent double continuation, unexpected cycles and exceptions interpreted as success. Security chains should deny unsupported paths deliberately rather than accidentally permit them.

INTERVIEW TRAP

Logging a rejection then continuing may undo the policy.

FOLLOW-UP EXPLAINED

Which handler can terminate processing?
Answer: The first handler that returns a handled/terminal result may stop the chain. Define errors and fallback behavior explicitly rather than relying on null.

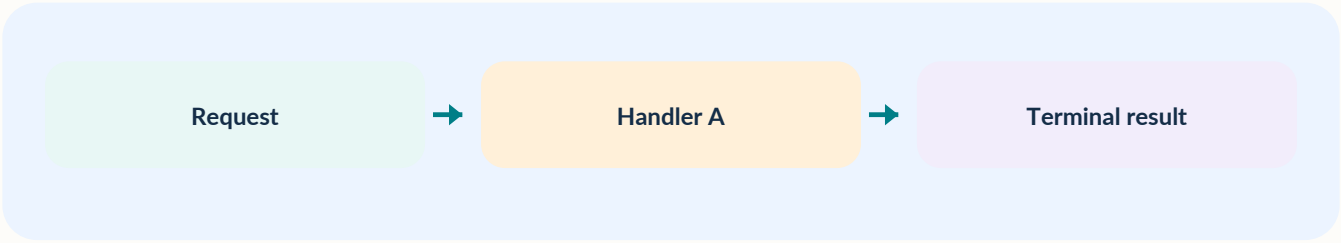
Q051 BEHAVIORAL

How does a chain differ from a fixed pipeline?

THE SHORT ANSWER

A chain emphasizes dynamic handling/propagation responsibility; a pipeline generally applies a defined sequence of transformations.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Handler may stop after processing
   Pipeline stage transforms input for the next stage
```

IN PLAIN ENGLISH

Implementations can overlap, so explain the actual continuation semantics. A servlet filter chain is a useful chain-style example, but Spring Security chain selection has its own first-match configuration rules.

INTERVIEW TRAP

One chain does not imply all configured chains run.

FOLLOW-UP EXPLAINED

How would you test handler order and short-circuiting?
Answer: Record handler invocations and assert the exact order. Add a case where handler two handles the request and confirm handler three never runs.

CHAPTER 16

Command

Represent an operation as a value

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q052 What is Command?

Q053 How can Command support undo?

Q054 How do commands need to change for distributed delivery?

INTERVIEW TIP

Use the Command pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

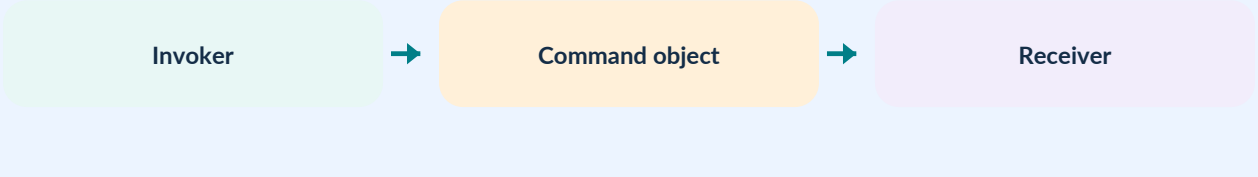
Q052 BEHAVIORAL

What is Command?

THE SHORT ANSWER

Command encapsulates a request, its receiver/context and execution behavior as an object.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface Command { void execute(); }
   record RenameCommand(Document doc, String title) implements Command {
     public void execute() { doc.setTitle(title); }
   }
```

IN PLAIN ENGLISH

It enables deferred invocation, queues, histories or audit metadata. A command object need not be remotely serializable or durable. Separate in-memory behavior from a persisted command envelope.

INTERVIEW TRAP

Capturing a live entity reference does not make a safe durable message.

FOLLOW-UP EXPLAINED

What must be stored for execution after restart?
Answer: Persist a durable command ID, validated payload, type/version, execution state, and retry policy. In-memory Runnable objects cannot survive a restart.

Q053 BEHAVIORAL

How can Command support undo?

THE SHORT ANSWER

Store the information needed to reverse a local action or provide an explicit compensating operation.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Rename records previousTitle before changing it
   undo restores that title under reviewed ordering rules
```

IN PLAIN ENGLISH

Not every action is reversible. Undo after another edit may overwrite valid work unless version/history ownership is defined. A payment refund is a new business action, not erasure of the original payment.

INTERVIEW TRAP

Undo is not a generic rollback of external side effects.

FOLLOW-UP EXPLAINED

What happens if compensation fails?
Answer: Escalate to reconciliation or a human recovery queue with the original operation ID. An undo command is not a guaranteed transaction rollback.

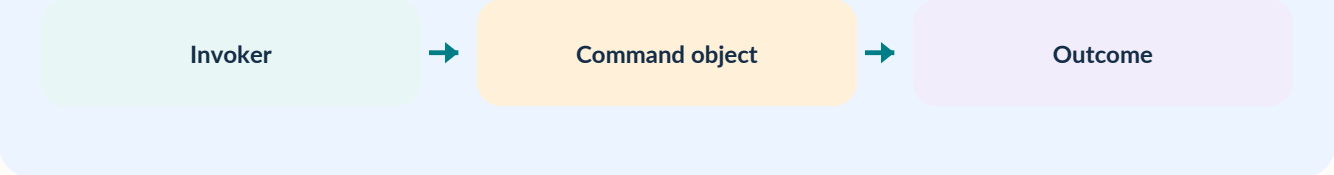
Q054 BEHAVIORAL

How do commands need to change for distributed delivery?

THE SHORT ANSWER

Use a stable command identity, serializable intent and a durable idempotent handling/recovery policy.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
ChargePayment(commandId, paymentId, amount)
```

IN PLAIN ENGLISH

A queued command may execute more than once. Deduplicate at the durable effect boundary or use a provider-supported idempotency key. Record accepted, executing and completed states if the workflow needs them.

INTERVIEW TRAP

GoF Command alone does not guarantee exactly-once execution.

FOLLOW-UP EXPLAINED

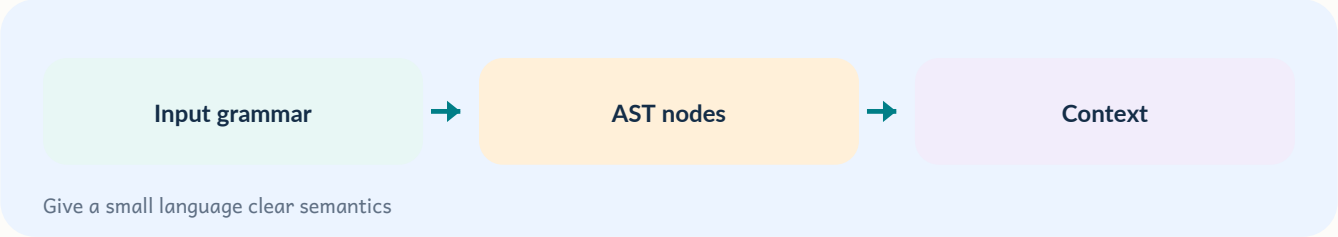
How do you handle an unknown payment outcome?
Answer: Store an idempotency key and query payment status before resubmitting. Treat unknown as pending until confirmed or reconciled, not as a definite failure.

CHAPTER 17

Interpreter

Give a small language clear semantics

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q055 What is Interpreter?

Q056 When should you choose a parser/library instead?

Q057 How do you keep an interpreter deterministic and testable?

INTERVIEW TIP

Use the Interpreter pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q055 BEHAVIORAL

What is Interpreter?

THE SHORT ANSWER

Interpreter represents grammar rules as objects that evaluate sentences in a supplied context.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Expr = Number | Add(Expr,Expr)
   Add(Number(2),Number(3)).eval() -> 5
```

IN PLAIN ENGLISH

Terminal nodes represent basic values; nonterminals combine expressions. It suits a small stable language or expression grammar. Parsing the text into the structure is a separate concern from evaluation.

INTERVIEW TRAP

An if statement is not automatically an Interpreter.

FOLLOW-UP EXPLAINED

Which part owns syntax errors?
Answer: The parser/grammar layer should identify malformed syntax with a useful position and safe message; execution handles valid syntax and its domain errors.

Q056 BEHAVIORAL

When should you choose a parser/library instead?

THE SHORT ANSWER

Choose a mature parser/evaluator when grammar, diagnostics, security or performance exceeds a small object-tree model.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Complex expression language -> grammar tool/library + controlled evaluation
```

IN PLAIN ENGLISH

The classical pattern can create many rule classes and expensive traversals. Do not expose arbitrary scripting capabilities merely to make business rules configurable. Bound nesting and evaluation cost for untrusted input.

INTERVIEW TRAP

A user expression can cause resource exhaustion even without code execution.

FOLLOW-UP EXPLAINED

How do you validate and limit rule complexity?
Answer: Restrict allowed operators, recursion depth, input length, evaluation time, and data access. Prefer a proven parser when grammar and error handling grow.

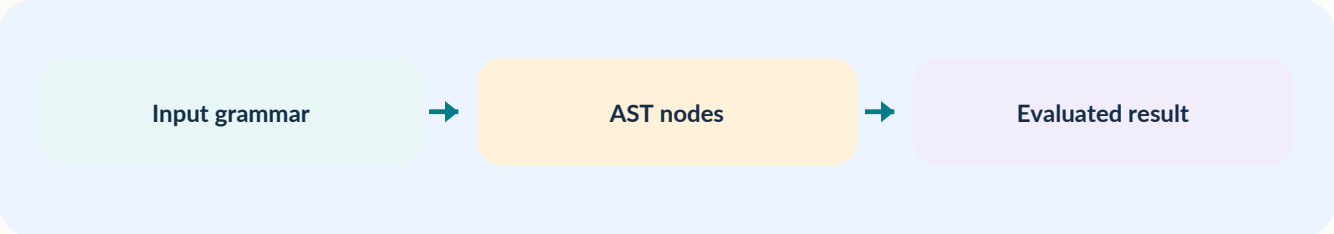
Q057 BEHAVIORAL

How do you keep an interpreter deterministic and testable?

THE SHORT ANSWER

Make evaluation depend on an explicit context and pure rules where possible, with errors represented consistently.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
boolean eval(Rule rule, Facts facts)
   // No hidden clock, network call or mutable global lookup.
```

IN PLAIN ENGLISH

Validate the grammar and supported operators. Define types, missing facts and comparison semantics. If time matters, inject a clock/fact rather than reading an invisible global time during traversal.

INTERVIEW TRAP

Different missing-value behavior can change policy outcomes.

FOLLOW-UP EXPLAINED

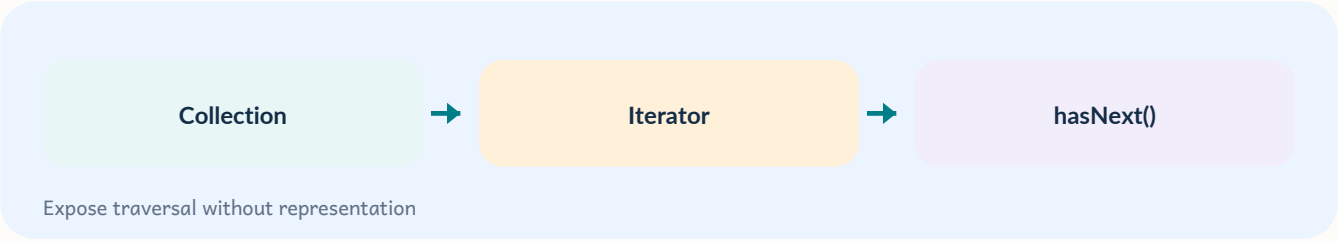
How would you explain a failed rule to a user?
Answer: Return the failed rule's stable identifier, safe input facts, and a readable explanation without exposing sensitive values or internal implementation details.

CHAPTER 18

Iterator

Expose traversal without representation

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q058 What is Iterator?

Q059 Are fail-fast iterators a concurrency guarantee?

Q060 How do Iterator and Stream differ?

INTERVIEW TIP

Use the Iterator pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q058 BEHAVIORAL

What is Iterator?

THE SHORT ANSWER

Iterator exposes sequential traversal while hiding the underlying aggregate representation.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Iterator<Order> it = orders.iterator();
   while (it.hasNext()) { process(it.next()); }
```

IN PLAIN ENGLISH

The traversal contract defines exhaustion and supported mutation. Java Iterable allows obtaining an iterator; Iterator carries traversal state. A tree or paged data source can provide a suitable traversal abstraction.

INTERVIEW TRAP

Iteration does not automatically mean all data is loaded or stable.

FOLLOW-UP EXPLAINED

What happens after exhaustion?
Answer: hasNext() becomes false; calling next() after exhaustion normally throws NoSuchElementException. Do not infer automatic restart.

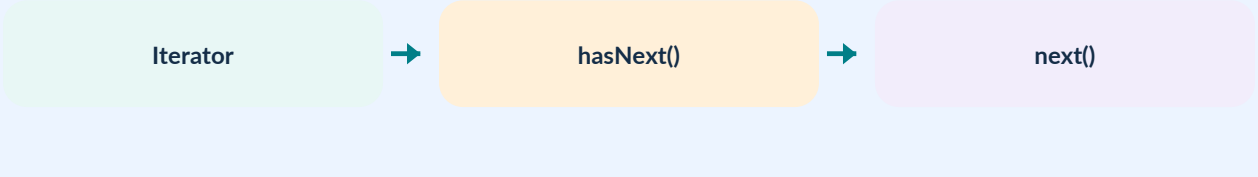
Q059 BEHAVIORAL

Are fail-fast iterators a concurrency guarantee?

THE SHORT ANSWER

No. Fail-fast detection is generally best effort and must not be used as a correctness mechanism for concurrent access.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Concurrent modification may throw ConcurrentModificationException
```

IN PLAIN ENGLISH

Choose synchronization, immutable snapshots or a collection with documented concurrent iteration semantics. Weakly consistent and snapshot iterators behave differently. Do not catch the exception and assume the traversal became complete.

INTERVIEW TRAP

No exception does not prove no concurrent modification occurred.

FOLLOW-UP EXPLAINED

Which semantics does CopyOnWriteArrayList provide?
Answer: CopyOnWriteArrayList iterators traverse a snapshot from iterator creation and do not reflect later writes; writes copy the array, making it suited to read-heavy cases.

Q060 BEHAVIORAL

How do Iterator and Stream differ?

THE SHORT ANSWER

Iterator gives explicit pull traversal; Stream describes a pipeline with its own consumption, laziness and resource semantics.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
orders.stream().filter(Order::paid).map(Order::id).toList();
```

IN PLAIN ENGLISH

A stream can be backed by a collection, iterator-like source or I/O resource. Streams are generally single-use and can require closing when resource-backed. Parallel execution adds ordering and thread-safety considerations.

INTERVIEW TRAP

Using Stream does not automatically make mutable work thread-safe.

FOLLOW-UP EXPLAINED

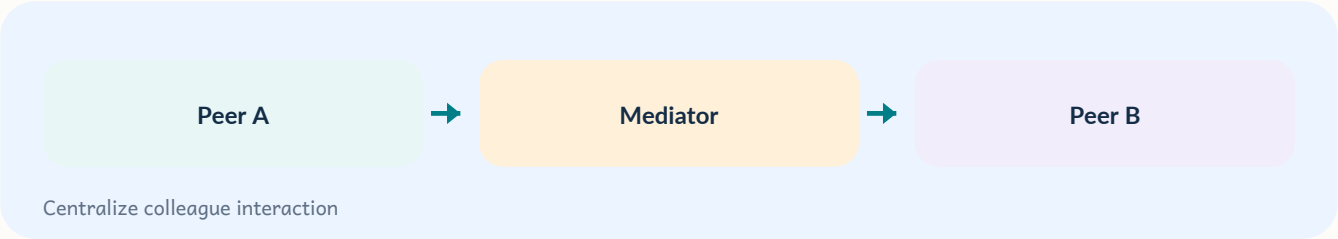
When is explicit iteration clearer?
Answer: Use an explicit loop when traversal needs early return, checked resource handling, mutation through Iterator.remove(), or step-by-step debugging.

CHAPTER 19

Mediator

Centralize colleague interaction

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q061** What is Mediator?
- **Q062** How does Mediator differ from Observer?
- **Q063** What are Mediator's main tradeoffs?

INTERVIEW TIP

Use the Mediator pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

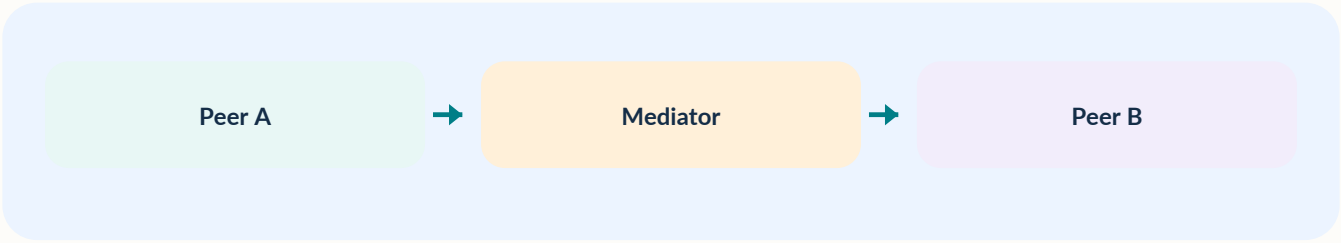
Q061 BEHAVIORAL

What is Mediator?

THE SHORT ANSWER

Mediator coordinates interactions among colleagues so they need not depend directly on every peer.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
FormMediator coordinates country, city and submit controls
```

IN PLAIN ENGLISH

A control reports a change to the mediator, which applies the agreed interaction rules. Domain work should remain with appropriate collaborators. This can reduce a dense dependency graph while making coordination more explicit.

INTERVIEW TRAP

A mediator that performs every task becomes a god object.

FOLLOW-UP EXPLAINED

Which rules should stay inside the controls?
Answer: Keep local validation and display rules inside each control. Put cross-control coordination, navigation, and dependency decisions in the Mediator.

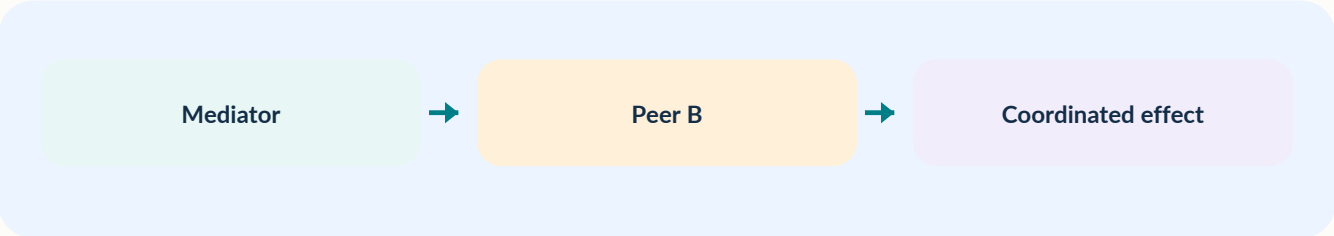
Q062 BEHAVIORAL

How does Mediator differ from Observer?

THE SHORT ANSWER

Mediator coordinates a specific interaction workflow; Observer distributes change notifications to subscribers.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Observer: country changed -> subscribers notified
   Mediator: country changed -> reload cities + reset selection + revalidate
```

IN PLAIN ENGLISH

A mediator can receive observer-style events. The distinction is the responsibility for coordination, not whether callbacks exist. Avoid circular event flows and define reentrancy/order where state changes trigger more notifications.

INTERVIEW TRAP

Event notification alone does not define workflow consistency.

FOLLOW-UP EXPLAINED

How would you prevent repeated update loops?
Answer: Use change detection, deduplication, or bounded propagation; do not let callbacks blindly publish the same update forever.

Q063 BEHAVIORAL

What are Mediator's main tradeoffs?

THE SHORT ANSWER

It reduces peer coupling but can concentrate complex coordination and become a bottleneck for changes.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Split checkout workflow and form workflow into coherent coordinators
```

IN PLAIN ENGLISH

Keep interaction rules cohesive, test transitions and outcomes, and avoid exposing every peer's internals. A mediator usually executes in-process; a distributed orchestrator needs durable state and recovery beyond this pattern.

INTERVIEW TRAP

An in-memory mediator cannot survive process failure by itself.

FOLLOW-UP EXPLAINED

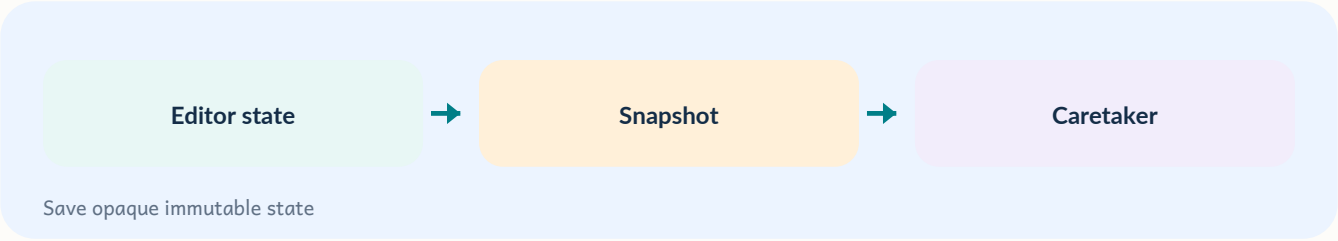
Which state needs persistence in a long-running workflow?
Answer: Persist workflow state, command IDs, completed steps, deadlines, and recovery data in durable storage. An in-memory mediator cannot resume after crash.

CHAPTER 20

Memento

Save opaque immutable state

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q064 What is Memento?

Q065 How is Memento different from Prototype?

Q066 How do you manage snapshot cost and compatibility?

INTERVIEW TIP

Use the Memento pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

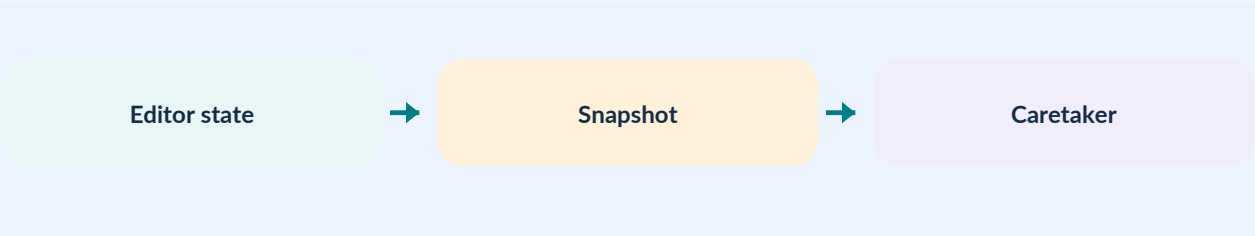
Q064 BEHAVIORAL

What is Memento?

THE SHORT ANSWER

Memento lets an originator capture and restore its state through an opaque snapshot managed by a caretaker.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Editor -> snapshot -> UndoHistory
   UndoHistory returns snapshot -> Editor restores
```

IN PLAIN ENGLISH

The originator understands the snapshot's internals; the caretaker stores it without rewriting state. Java access control and immutable snapshot types can support the boundary. Choose the required state depth.

INTERVIEW TRAP

Exposing a mutable live list is not a safe snapshot.

FOLLOW-UP EXPLAINED

Which fields belong in the snapshot?
Answer: Save only state needed to restore the defined editor behavior: content, cursor, and selected options. Avoid copying unrelated services or credentials.

Q065 BEHAVIORAL

How is Memento different from Prototype?

THE SHORT ANSWER

Memento captures restoration state for an existing originator; Prototype creates a separate configured object by copying.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Memento: restore editor before last edit
   Prototype: create a new document from a template
```

IN PLAIN ENGLISH

Both involve copying, but identity and intent differ. A snapshot may exclude derived/transient fields, while a prototype may reset IDs and retain configuration. Avoid duplicating resources as plain data.

INTERVIEW TRAP

Returning the same mutable reference defeats both intents.

FOLLOW-UP EXPLAINED

Which identity survives restoration?
Answer: The editor keeps its business identity; restoring a Memento changes its state. Prototype usually makes a new object with a new identity.

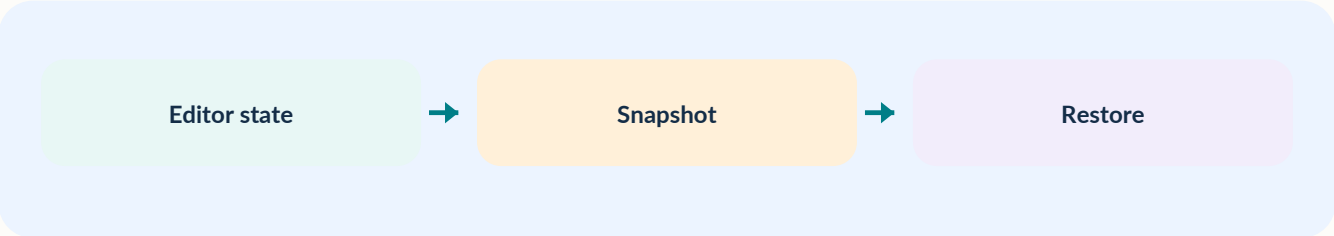
Q066 BEHAVIORAL

How do you manage snapshot cost and compatibility?

THE SHORT ANSWER

Bound history, use immutable/shared data safely, and version snapshots when restoring across deployments.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Keep last 50 editor states; discard older history
```

IN PLAIN ENGLISH

Large graphs make full snapshots expensive. Deltas reduce size but complicate replay and dependency ordering. Do not serialize secrets or resource handles casually. Domain restoration must still enforce valid current invariants.

INTERVIEW TRAP

A historical snapshot may no longer match the current schema.

FOLLOW-UP EXPLAINED

When would an event log be more suitable?
Answer: Use event logs when you need durable audit, rebuilds, and a history of changes rather than only point-in-time editor undo. Replay and versioning must be designed.

CHAPTER 21

Observer

Notify subscribers with explicit lifecycle

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q067 What is Observer?

Q068 How do you avoid observer leaks and reentrancy problems?

Q069 Should you use java.util.Observable for new Java code?

INTERVIEW TIP

Use the Observer pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

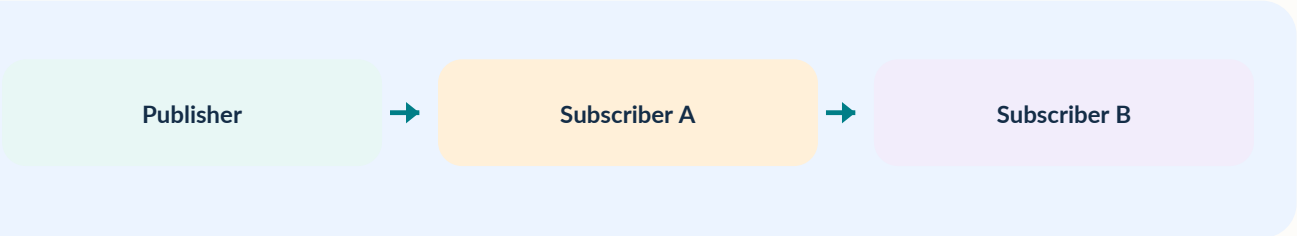
Q067 BEHAVIORAL

What is Observer?

THE SHORT ANSWER

Observer lets subscribers receive notifications when a subject's state or events change.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
subject.subscribe(auditListener)
   subject.publish(new OrderPlaced("42"))
```

IN PLAIN ENGLISH

The subject depends on a notification contract rather than concrete subscribers. Synchronous and asynchronous delivery are separate execution choices. Document callback order and errors if they matter.

INTERVIEW TRAP

Observer alone is not a durable message broker.

FOLLOW-UP EXPLAINED

What happens when one subscriber throws?
Answer: A synchronous publisher may propagate the exception and stop later listeners unless it isolates them. Decide whether each subscriber failure is critical.

Q068 BEHAVIORAL

How do you avoid observer leaks and reentrancy problems?

THE SHORT ANSWER

Make subscription lifetime explicit and prevent uncontrolled modification/recursive notification during callbacks.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Subscription s = subject.subscribe(listener);
   s.close(); // remove when no longer needed
```

IN PLAIN ENGLISH

A subject holding strong references can retain subscribers indefinitely. Snapshotting the subscriber list can simplify iteration but does not make callback state thread-safe. Define whether callbacks can subscribe/unsubscribe or publish recursively.

INTERVIEW TRAP

Weak references are not a universal delivery/lifecycle solution.

FOLLOW-UP EXPLAINED

How would you test nested notification?
Answer: Register a listener that publishes another event; assert bounded order, no duplicate loop, correct removal, and cleanup after exceptions.

Q069 BEHAVIORAL

Should you use java.util.Observable for new Java code?

THE SHORT ANSWER

Usually prefer an explicit event/listener contract or an appropriate reactive/framework API; Observable and Observer are deprecated.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface OrderListener { void on(OrderPlaced event); }
```

IN PLAIN ENGLISH

The older APIs impose limitations and weak event typing. A custom contract makes ownership and failure behavior visible. Spring events provide application-context integration, but reliability and transaction timing remain configuration choices.

INTERVIEW TRAP

Deprecation of the class does not invalidate the Observer pattern.

FOLLOW-UP EXPLAINED

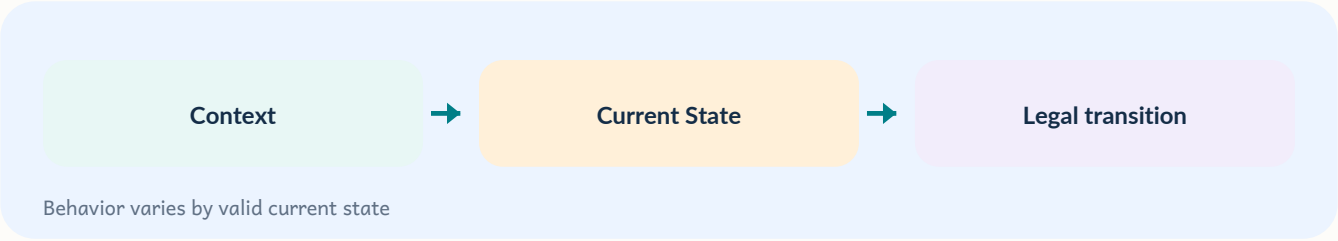
When is a durable event transport required?
Answer: Use a durable broker/outbox when events must survive crashes, support independent consumers, and permit replay/retry. In-process Observer has none of those guarantees.

CHAPTER 22

State

Behavior varies by valid current state

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q070 What is State?

Q071 When is a state table simpler than State classes?

Q072 How do you implement State safely for persisted workflows?

INTERVIEW TIP

Use the State pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

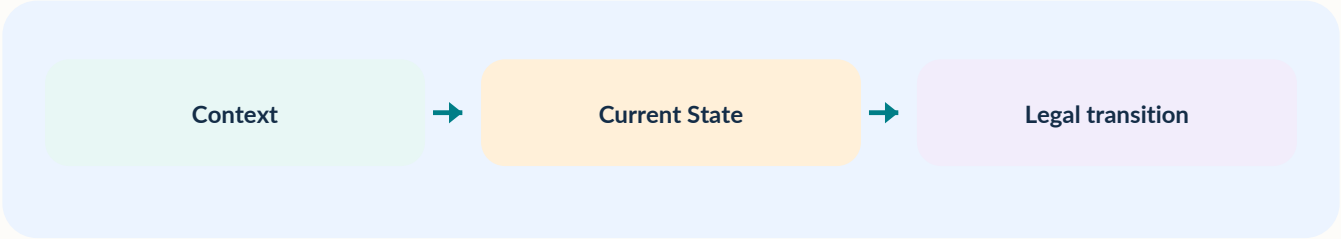
Q070 BEHAVIORAL

What is State?

THE SHORT ANSWER

State delegates behavior to an object representing the context's current state, allowing behavior to change with transitions.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
OrderState current = PendingState;
   current.cancel(order); // allowed only by its state contract
```

IN PLAIN ENGLISH

A state can reject invalid actions or transition to another state. The context and states divide transition ownership explicitly. State is useful when many operations have substantial state-specific behavior.

INTERVIEW TRAP

An enum label alone is not necessarily the State pattern.

FOLLOW-UP EXPLAINED

Who decides the next state?
Answer: An explicit transition policy or current State implementation decides the next state; persisted workflows should validate and atomically store that transition.

Q071 BEHAVIORAL

When is a state table simpler than State classes?

THE SHORT ANSWER

Use a table or small switch when transitions are simple data and behavior variation is limited.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
PENDING + PAY -> PAID
   PAID + SHIP -> SHIPPED
   SHIPPED + CANCEL -> reject
```

IN PLAIN ENGLISH

A table makes allowed transitions easy to audit. State classes help when several operations need rich behavior in each state. Neither automatically handles concurrent transitions or persistence.

INTERVIEW TRAP

Every status enum does not require one class per value.

FOLLOW-UP EXPLAINED

How would you protect a transition against concurrent updates?
Answer: Use compare-and-set on a version field or an atomic SQL update guarded by current state to reject two conflicting approvals.

Q072 BEHAVIORAL

How do you implement State safely for persisted workflows?

THE SHORT ANSWER

Enforce transitions against durable version/current-state constraints and handle duplicate or out-of-order events.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
UPDATE orders SET status='PAID', version=version+1
   WHERE id=:id AND status='PENDING' AND version=:expected;
```

IN PLAIN ENGLISH

State objects can express policy, while the database enforces the actual transition under concurrency. Record event identity when duplicate delivery matters. A transition that emits an event needs reliable publication such as an outbox.

INTERVIEW TRAP

An in-memory state change cannot guarantee the database committed.

FOLLOW-UP EXPLAINED

What if payment succeeds before persistence fails?
Answer: Record payment intent/outcome with an idempotency key and reconcile after a crash. Never assume a failed DB commit undoes an external payment.

CHAPTER 23

Strategy

Swap algorithms behind shared contract

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q073 What is Strategy?

Q074 How should strategy selection be implemented?

Q075 How do you test Strategies for substitutability?

INTERVIEW TIP

Use the Strategy pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q073 BEHAVIORAL

What is Strategy?

THE SHORT ANSWER

Strategy encapsulates interchangeable algorithms behind a contract used by a context.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface Discount { long apply(long cents); }
   Discount tenPercent = cents -> cents - cents / 10;
```

IN PLAIN ENGLISH

The context delegates without encoding every algorithm's rules. Lambdas fit single-method policies, while classes fit richer state/configuration. Define precision, valid inputs and output guarantees so strategies remain interchangeable.

INTERVIEW TRAP

Algorithm interchangeability does not excuse different business units or semantics.

FOLLOW-UP EXPLAINED

How would you validate a new strategy?
Answer: Run shared contract tests, boundary values, invalid identifiers, and golden examples; confirm it obeys the same return and error policy as other strategies.

Q074 BEHAVIORAL

How should strategy selection be implemented?

THE SHORT ANSWER

Choose through an explicit policy/registry and reject unsupported identifiers deliberately.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Discount d = strategies.get(code);
   if (d == null) throw new IllegalArgumentException("Unknown discount");
```

IN PLAIN ENGLISH

Selection and execution are distinct responsibilities. A small switch at the composition boundary can be perfectly reasonable. In Spring, injected strategies can be assembled into a validated map without reflection.

INTERVIEW TRAP

Silent fallback can apply the wrong price or authorization policy.

FOLLOW-UP EXPLAINED

What if two strategies claim the same identifier?
Answer: Fail startup on duplicate IDs in the registry rather than choose by iteration order; explicit configuration should select the single intended strategy.

Q075 BEHAVIORAL

How do you test Strategies for substitutability?

THE SHORT ANSWER

Run shared contract tests across implementations, then test each algorithm's specific examples and boundaries.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
For valid cents: result >= 0 and result <= original
   Then verify agreed rounding for each discount
```

IN PLAIN ENGLISH

The contract should reflect real business rules, not invented constraints that every policy cannot satisfy. Stateful strategies need lifecycle and concurrency review. Compare outputs using exact units appropriate to the domain.

INTERVIEW TRAP

A shared interface does not prove behavioral compatibility.

FOLLOW-UP EXPLAINED

Which extreme inputs reveal overflow?
Answer: Test zero, negative (if forbidden), Long.MAX_VALUE, arithmetic overflow, tiny rounding amounts, nulls, and unsupported policies.

CHAPTER 24

Template Method

Freeze skeleton and vary limited steps

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

- **Q076** What is Template Method?
- **Q077** How do hooks differ from abstract steps?
- **Q078** When should Template Method become Strategy composition?

INTERVIEW TIP

Use the Template Method pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q076 BEHAVIORAL

What is Template Method?

THE SHORT ANSWER

Template Method fixes an algorithm's sequence in a superclass and lets subclasses specialize selected steps.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
abstract class ImportJob {
     final void run() { read(); validate(); save(); }
     abstract void read(); abstract void validate(); abstract void save();
   }
```

IN PLAIN ENGLISH

The template owns ordering, while steps provide variation. Making the template final can protect sequencing. This is suited to a stable shared workflow, not arbitrary independently combinable behaviors.

INTERVIEW TRAP

Subclasses can still violate step contracts even when run is final.

FOLLOW-UP EXPLAINED

Which invariant depends on the step order?
Answer: The order may ensure validate before charge and cleanup after resources are opened. Tests must show reordered steps would violate a stated invariant.

Q077 BEHAVIORAL

How do hooks differ from abstract steps?

THE SHORT ANSWER

Abstract steps require implementation; hooks provide optional extension points with defined defaults.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
protected void beforeSave() { } // optional no-op hook
```

IN PLAIN ENGLISH

Hooks should be few and have precise timing/side-effect contracts. Too many hooks create an implicit framework that is difficult to reason about. Template execution must define cleanup if a step fails.

INTERVIEW TRAP

A finally block should not hide an earlier failure with cleanup errors.

FOLLOW-UP EXPLAINED

What resources must be released after read fails?
Answer: Close streams, files, and locks in finally or try-with-resources even when reading fails. Avoid leaving the resource alive after a subclass hook throws.

Q078 BEHAVIORAL

When should Template Method become Strategy composition?

THE SHORT ANSWER

Refactor when algorithm steps vary independently, runtime selection matters or inheritance causes many combinations.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Import workflow injects Parser, Validator and Writer
```

IN PLAIN ENGLISH

Composition makes independent dimensions explicit and easier to replace in tests. A template remains reasonable if the subclass is the meaningful unit of specialization. Keep sequencing policy in one place whichever mechanism you choose.

INTERVIEW TRAP

Replacing inheritance with arbitrary callbacks can obscure invariants.

FOLLOW-UP EXPLAINED

Who owns the workflow after refactoring?
Answer: The composed workflow/orchestrator owns step order and lifecycle; injected Strategy objects own only their interchangeable algorithm behavior.

CHAPTER 25

Visitor

Add operations to a stable type family

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q079 What is Visitor?

Q080 What is Visitor's extension tradeoff?

Q081 How do Java's newer features affect Visitor choices?

INTERVIEW TIP

Use the Visitor pattern only when the changing responsibility is real. State the benefit, cost, and failure case.

Q079 BEHAVIORAL

What is Visitor?

THE SHORT ANSWER

Visitor separates operations from elements by letting elements dispatch to type-specific visitor methods.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
interface Expr { <R> R accept(Visitor<R> v); }
   // NumberExpr.accept -> v.visitNumber(this)
   // AddExpr.accept -> v.visitAdd(this)
```

IN PLAIN ENGLISH

The element call and overloaded visit method establish the classic double-dispatch structure. Evaluation, printing or analysis can be separate visitors over the same tree. The tree structure stays with elements.

INTERVIEW TRAP

Overloading alone does not dynamically select by runtime parameter type.

FOLLOW-UP EXPLAINED

Why is accept needed?
Answer: accept(this) allows a node's runtime class to select the correct visit(NodeType) method, achieving double dispatch without repeated instanceof chains.

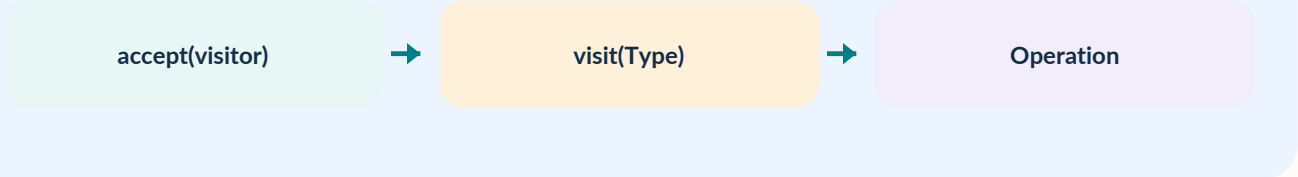
Q080 BEHAVIORAL

What is Visitor's extension tradeoff?

THE SHORT ANSWER

Adding an operation is relatively easy; adding an element type can require updating every visitor.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
New operation: PrettyPrinter visitor
   New element: MultiplyExpr -> update Visitor and implementations
```

IN PLAIN ENGLISH

Visitor fits a stable hierarchy with frequently added operations. If element kinds grow often, polymorphic methods or a version-appropriate sealed-type dispatch may be simpler. Avoid default handlers that silently skip unsupported elements.

INTERVIEW TRAP

The visitor can become brittle as the element set expands.

FOLLOW-UP EXPLAINED

Which dimension changes more often here?
Answer: If operations change frequently but node types stay stable, Visitor fits; if new node types arrive often, adding methods to each visitor becomes expensive.

Q081 BEHAVIORAL

How do Java's newer features affect Visitor choices?

THE SHORT ANSWER

Sealed hierarchies and version-supported pattern matching can make some closed-type dispatch concise, but they do not erase Visitor's design intent.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Java 17: sealed types can model a closed element set
   Pattern-switch examples require the relevant Java version/status
```

IN PLAIN ENGLISH

This guide's executable-style examples use ordinary Java 17 constructs without preview features. Choose Visitor when operation objects, traversal or APIs benefit from it. Choose simpler dispatch when the closed hierarchy and operation set stay small.

INTERVIEW TRAP

Do not present newer nonpreview syntax as portable Java 17.

FOLLOW-UP EXPLAINED

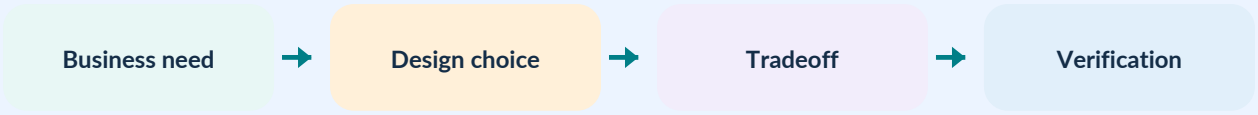
Would an ordinary method on Expr be enough?
Answer: Yes, if an operation naturally belongs to Expr and does not need many independent traversals, a regular method can be simpler than Visitor.

CHAPTER 26

Patterns that interviewers compare

Name the intent and difference, even when class diagrams look alike.

ROLE DIAGRAM / THE MAIN COLLABORATION



Name the intent and difference, even when class diagrams look alike.

WHAT TO MASTER

- **Q082** How do Factory Method and Abstract Factory differ?
- **Q083** How do Strategy and State differ?
- **Q084** How do Adapter, Decorator and Proxy differ?

INTERVIEW TIP

Explain the actual domain constraint first, then name the smallest pattern that preserves it.

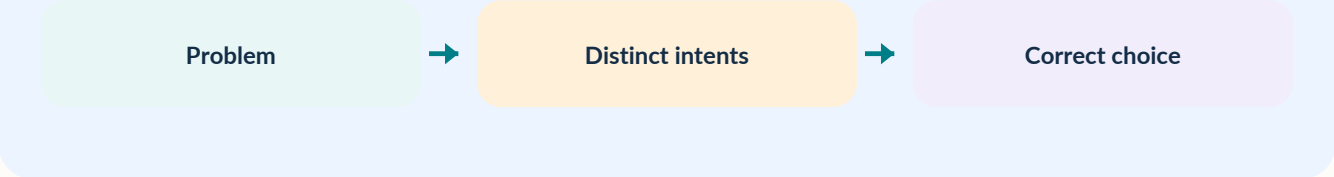
Q082 ADVANCED

How do Factory Method and Abstract Factory differ?

THE SHORT ANSWER

Factory Method varies product creation through a creator hook; Abstract Factory creates a family of related products through a factory contract.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Factory Method: ImportJob.createParser()
   Abstract Factory: UiFactory.button() + menu()
```

IN PLAIN ENGLISH

Abstract Factory can itself be implemented using factory methods. Compare family compatibility and which dimension changes, rather than claiming the patterns cannot coexist. Static factory methods are another distinct idiom.

INTERVIEW TRAP

The method name create is not enough to identify either GoF pattern.

FOLLOW-UP EXPLAINED

What changes when a new product kind is introduced?
Answer: A new product kind normally requires updating the AbstractFactory interface and each family; Factory Method may add another creator variation separately.

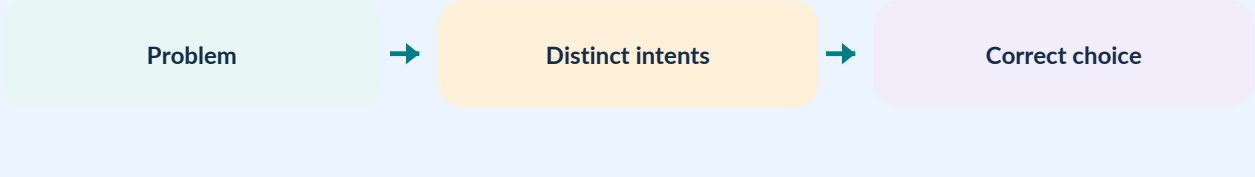
Q083 ADVANCED

How do Strategy and State differ?

THE SHORT ANSWER

Strategy models a selected algorithm; State models behavior tied to the context's state and transitions.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Strategy: choose pricing policy
   State: pending order can pay; shipped order rejects cancel
```

IN PLAIN ENGLISH

Both delegate through interfaces. Clients typically select strategies, while context/state logic governs transitions, though implementations vary. A state transition has lifecycle meaning beyond swapping any algorithm.

INTERVIEW TRAP

Similar delegation diagrams do not imply identical intent.

FOLLOW-UP EXPLAINED

Would replacing a discount policy count as an order transition?
Answer: No. Choosing a DiscountPolicy is usually a Strategy selection; changing an order from PENDING to APPROVED is a state transition with legal-state rules.

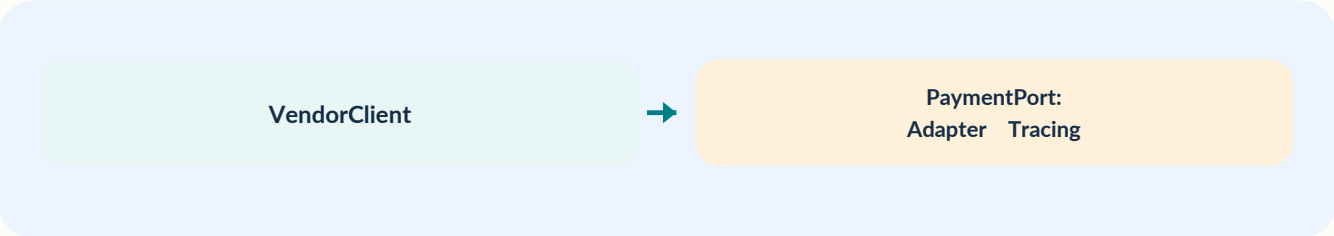
Q084 ADVANCED

How do Adapter, Decorator and Proxy differ?

THE SHORT ANSWER

Adapter translates an interface; Decorator adds composable responsibilities; Proxy controls subject access.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
VendorClient -> PaymentPort: Adapter
   TracingSender wraps Sender: Decorator
   AuthorizedReport wraps Report: Proxy
```

IN PLAIN ENGLISH

The same wrapper class may serve more than one purpose, but describe the primary contract and effect. All require careful error, identity and lifecycle semantics. A class name alone is weak evidence.

INTERVIEW TRAP

Same-interface wrapping can still violate substitutability.

FOLLOW-UP EXPLAINED

Which concern is visible to the client?
Answer: Adapter changes the interface, which clients may notice; Decorator and Proxy preserve a subject contract but expose added behavior or access rules.

Q085 ADVANCED

How do Builder and Abstract Factory differ?

THE SHORT ANSWER

Builder assembles a complex representation through steps; Abstract Factory creates compatible members of a family.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
ReportBuilder: header, sections, footer -> Report
   UiFactory: Button + Menu -> compatible family
```

IN PLAIN ENGLISH

A builder can use an abstract factory to create parts, and a director can drive construction. Compare construction sequencing versus family compatibility. A fluent API does not automatically need a director.

INTERVIEW TRAP

Not every class returning several objects is a Builder.

FOLLOW-UP EXPLAINED

Which validation occurs at build time?
Answer: Builder validates final fields and combinations at build time. Abstract Factory chooses related product implementations; each product may validate itself.

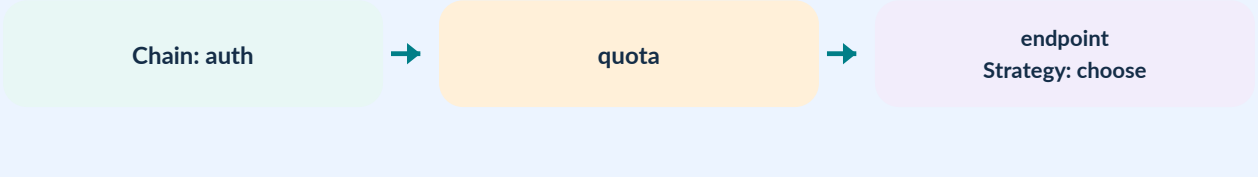
Q086 ADVANCED

How do Chain, Strategy and Command differ?

THE SHORT ANSWER

Chain routes processing among handlers; Strategy supplies an algorithm; Command represents a requested action.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Chain: auth -> quota -> endpoint
   Strategy: choose quota algorithm
   Command: queued RebuildProjection request
```

IN PLAIN ENGLISH

The designs can coexist in one use case. A chain handler might invoke a selected strategy or execute a command. The important distinction is routing, algorithm variation or request identity/lifecycle.

INTERVIEW TRAP

Queueing a Strategy does not by itself define a durable Command protocol.

FOLLOW-UP EXPLAINED

Which object should carry the requestId?
Answer: Put requestId or idempotency ID on the Command payload that represents the logical request; the selected handler/strategy uses it consistently.

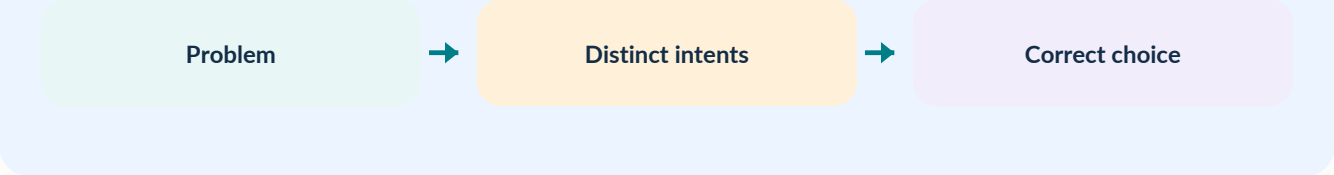
Q087 ADVANCED

How do Decorator and inheritance compare for feature combinations?

THE SHORT ANSWER

Decorator composes behaviors at assembly time; inheritance encodes combinations in class relationships.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Retry(Trace(Sender)) instead of RetryTraceSender subclass
```

IN PLAIN ENGLISH

Composition avoids many subclasses but makes order and lifecycle visible obligations. Inheritance can be clearer for a few stable specializations with a genuine subtype contract. Prefer the structure matching actual variation and testing needs.

INTERVIEW TRAP

A stack of wrappers can be harder to debug than one coherent class.

FOLLOW-UP EXPLAINED

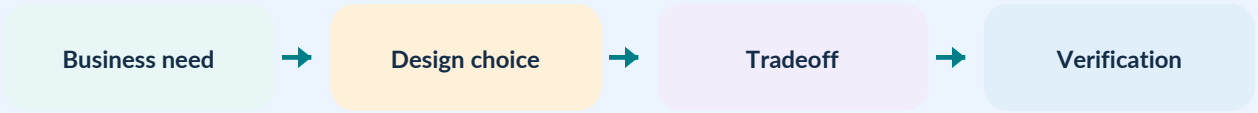
How many combinations truly exist?
Answer: Count actual combinations of independent features (for example, 3 channels x 4 behaviors). Decorators are useful when valid combinations are numerous.

CHAPTER 27

Patterns in Java and Spring

Recognize framework collaborations without assuming hidden guarantees.

ROLE DIAGRAM / THE MAIN COLLABORATION



Recognize framework collaborations without assuming hidden guarantees.

WHAT TO MASTER

Q088 Is Spring singleton scope the GoF Singleton?

Q089 How do Spring factories relate to GoF creation patterns?

Q090 Where do Strategy and Template Method appear in Spring-style c...

INTERVIEW TIP

Explain the actual domain constraint first, then name the smallest pattern that preserves it.

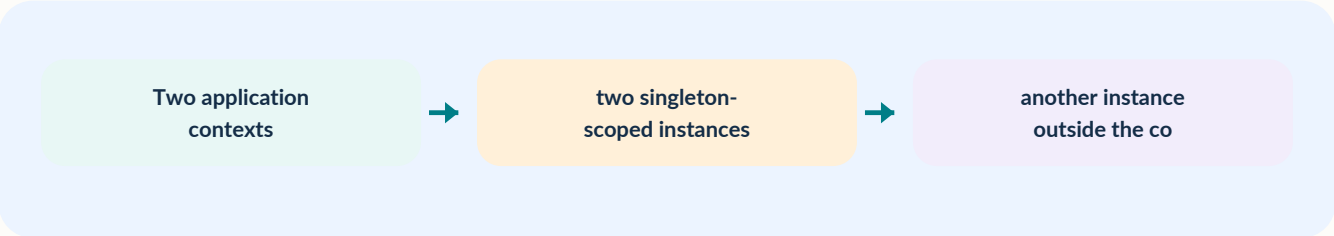
Q088 ADVANCED

Is Spring singleton scope the GoF Singleton?

THE SHORT ANSWER

No. Spring singleton scope is normally one bean instance per bean definition per container, rather than a globally enforced unique Java instance.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Two application contexts -> two singleton-scoped instances
   Manual new -> another instance outside the container
```

IN PLAIN ENGLISH

A stateless singleton-scoped service is a useful lifecycle choice. Mutable fields still need concurrency design. The container makes dependency replacement easier than static global lookup.

INTERVIEW TRAP

Singleton scope does not make request-specific fields safe.

FOLLOW-UP EXPLAINED

How would you store per-request data instead?
Answer: Store per-request data in local variables, request-scoped beans, or explicit immutable request context; never put mutable user data on a shared singleton service.

Q089 ADVANCED

How do Spring factories relate to GoF creation patterns?

THE SHORT ANSWER

BeanFactory, FactoryBean and @Bean methods manage creation/lifecycle, but their GoF classification depends on the actual intent and collaboration.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
FactoryBean<T>.getObject() -> exposes a managed product
   @Bean method -> composition/configuration point
```

IN PLAIN ENGLISH

A FactoryBean is itself a bean, with a distinct product-access convention. Do not claim every @Bean method is the inheritance-based Factory Method. Framework names and pattern names are not interchangeable definitions.

INTERVIEW TRAP

Retrieving a FactoryBean product differs from retrieving the factory itself.

FOLLOW-UP EXPLAINED

What does the & bean-name prefix mean?
Answer: In Spring, getBean("&name") fetches the FactoryBean itself; getBean("name") usually fetches its produced object.

Q090 ADVANCED

Where do Strategy and Template Method appear in Spring-style code?

THE SHORT ANSWER

Injected policies often use Strategy; skeletal frameworks may use template workflows and callbacks for controlled variation.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
OrderService injects PaymentPort implementations
   JdbcTemplate centralizes JDBC resource/error workflow
```

IN PLAIN ENGLISH

JdbcTemplate's public callback API demonstrates template/callback organization; avoid forcing a pure subclass-only GoF classification onto every method. Name the stable skeleton and variation point. Application strategy selection should remain explicit and validated.

INTERVIEW TRAP

A class ending Template does not mean users must subclass it.

FOLLOW-UP EXPLAINED

Which work does JdbcTemplate own?
Answer: JdbcTemplate owns connection/resource handling, execution skeleton, and cleanup; the callback supplies SQL-specific value extraction or binding behavior.

Q091 ADVANCED

Where do Proxy and Adapter appear in Spring MVC and AOP?

THE SHORT ANSWER

AOP proxies intercept access to advised beans; MVC HandlerAdapter adapts supported handler forms to the dispatcher's invocation needs.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Caller -> AOP proxy -> advised service
   DispatcherServlet -> HandlerAdapter -> supported handler
```

IN PLAIN ENGLISH

Both examples have concrete framework contracts. Proxy type and advice behavior depend on configuration; MVC adapters decide support and invocation. These are framework mechanisms, not automatic guarantees of business atomicity.

INTERVIEW TRAP

A controller invoking itself does not gain an external proxy crossing.

FOLLOW-UP EXPLAINED

Which boundary does a handler adapter translate?
Answer: A Spring MVC HandlerAdapter translates a chosen handler contract into an invocation that DispatcherServlet can use; an AOP proxy controls advised method calls.

Q092 ADVANCED

Are Spring application events durable Observer delivery?

THE SHORT ANSWER

No. Application events integrate publishers/listeners inside the application context; execution and transaction timing depend on configuration.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
publisher.publishEvent(new OrderPlaced("42"));
```

IN PLAIN ENGLISH

A default event multicaster commonly invokes synchronously, while asynchronous setup changes propagation and failure behavior. A transactional event listener can align handling with a transaction phase, but is not a durable cross-service outbox.

INTERVIEW TRAP

Process failure can lose work held only in memory.

FOLLOW-UP EXPLAINED

How would you publish the order outcome reliably?
Answer: Persist an outbox row in the same DB transaction as the order, then publish asynchronously with retries and idempotent consumers. Plain in-process events are not durable.

Q093 ADVANCED

How do repository and hexagonal ports complement GoF patterns?

THE SHORT ANSWER

They isolate domain access and dependency direction at architectural boundaries; adapters and strategies can implement those boundaries.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Domain OrderRepository port <- JpaOrderAdapter
   Domain PaymentPort <- VendorPaymentAdapter
```

IN PLAIN ENGLISH

Repository is not one of the 23 GoF entries. A domain-facing contract should avoid leaking infrastructure types when isolation matters. Choose the boundary from business requirements rather than wrapping every API mechanically.

INTERVIEW TRAP

A new interface that exposes EntityManager does not isolate JPA.

FOLLOW-UP EXPLAINED

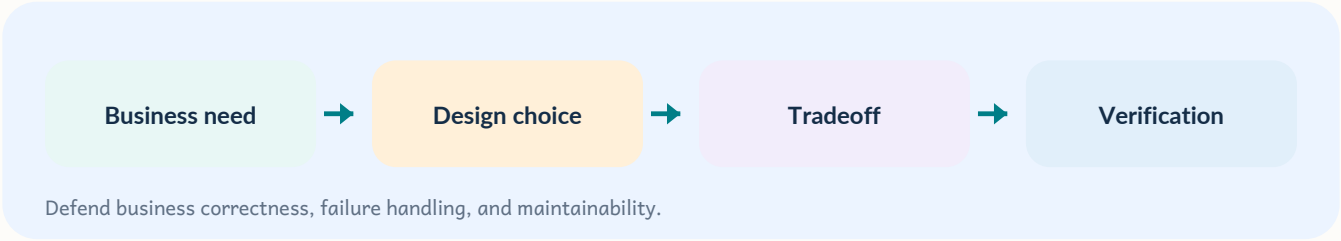
What should remain in the domain package?
Answer: Keep business rules, domain models, and ports in the domain/application core. Put concrete Spring Data repositories and HTTP adapters outside.

CHAPTER 28

Senior interview scenarios and anti-patterns

Defend business correctness, failure handling, and maintainability.

ROLE DIAGRAM / THE MAIN COLLABORATION



WHAT TO MASTER

Q094 A service has a huge payment switch. How would you refactor it?

Q095 How would you design configurable notification channels?

Q096 How would you design a loan approval workflow?

INTERVIEW TIP

Explain the actual domain constraint first, then name the smallest pattern that preserves it.

Q094 ADVANCED

A service has a huge payment switch. How would you refactor it?

THE SHORT ANSWER

Separate provider selection from payment policy and translate each provider through a stable port where variation is real.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Registry selects PaymentPort by provider code
   Vendor adapters map requests and outcomes
```

IN PLAIN ENGLISH

Keep common orchestration in one coherent use case; use Strategy-like selection and Adapter boundaries as needed. Reject unknown providers and validate duplicate registrations. Preserve idempotency and uncertain-outcome handling through the refactor.

INTERVIEW TRAP

Removing a switch is not enough if each implementation changes the contract.

FOLLOW-UP EXPLAINED

How would you prove behavior stayed compatible?
Answer: Run characterization/contract tests on old and new implementations for all payment types, including timeout, amount rounding, duplicate requests, and decline handling.

Q095 ADVANCED

How would you design configurable notification channels?

THE SHORT ANSWER

Use a small channel contract with explicit routing and adapters; add wrappers only for required cross-cutting behavior.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
NotificationService selects Channel
   Email/Sms adapters implement Channel
   Tracing decorator records each logical attempt
```

IN PLAIN ENGLISH

If notification kinds and channel implementations vary independently, Bridge can organize those dimensions. Define retries, duplicate protection and recipient validation separately. A send interface should expose actual failure semantics.

INTERVIEW TRAP

A wrapper named Retry does not make sending safe to repeat.

FOLLOW-UP EXPLAINED

What identity would deduplicate a notification?
Answer: Use an immutable notificationId representing one logical send. Store a per-channel delivery result to suppress duplicate work across retries.

Q096 ADVANCED

How would you design a loan approval workflow?

THE SHORT ANSWER

Make durable states and allowed transitions explicit, then use replaceable policies for eligibility and handlers for ordered checks when justified.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Application state: RECEIVED -> VERIFIED -> APPROVED or REJECTED
   EligibilityPolicy selected by product
```

IN PLAIN ENGLISH

State governs workflow lifecycle; Strategy governs decision algorithm variation. Persist versioned transitions and record decisions for audit. An in-memory mediator or command object alone cannot recover a long-running process.

INTERVIEW TRAP

A design pattern does not replace durable orchestration requirements.

FOLLOW-UP EXPLAINED

How do you recover after a provider callback arrives twice?
Answer: Store callback/event IDs and a versioned loan state transition in durable storage. On duplicate callback, return the prior result without approving twice.

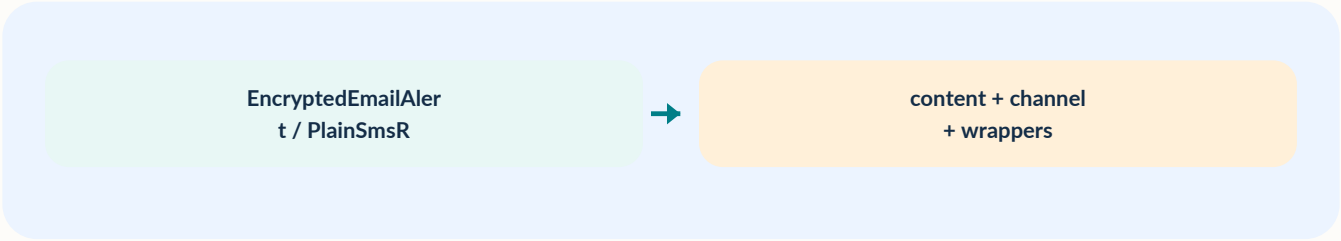
Q097 ADVANCED

How do you refactor inheritance that has too many combinations?

THE SHORT ANSWER

Identify independently varying dimensions, then extract stable contracts and compose them incrementally.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
EncryptedEmailAlert / PlainSmsReceipt -> content + channel + wrappers
```

IN PLAIN ENGLISH

Use characterization/contract tests around meaningful behavior before changing structure. Bridge can split axes and Decorator can compose optional concerns, but only if those roles match requirements. Migrate callers without changing outcomes unnecessarily.

INTERVIEW TRAP

Extracting dozens of tiny interfaces can preserve the original complexity.

FOLLOW-UP EXPLAINED

Which axis would you separate first?
Answer: Separate whichever axis changes independently most often - for example transport from notification content - while preserving current public behavior.

Q098 ADVANCED

How should patterns handle concurrency and resource ownership?

THE SHORT ANSWER

Define immutability, thread-safety and close/lifecycle rules explicitly; the pattern name provides none of them automatically.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Shared Strategy: stateless or safely synchronized
   Decorator: owns or borrows wrapped resource by contract
```

IN PLAIN ENGLISH

Singletons, pools, observers and mutable commands need especially careful review. Prefer immutable values and per-operation state where feasible. Test realistic concurrent use and failure cleanup rather than only a happy-path class diagram.

INTERVIEW TRAP

Composition does not prevent data races or double-close.

FOLLOW-UP EXPLAINED

Which object is responsible for shutdown?
Answer: The object or application scope that creates executors, connections, and wrapper stacks owns shutdown; define close order and ensure it runs even after failure.

Q099 ADVANCED

What anti-patterns should you recognize in interviews?

THE SHORT ANSWER

Look for god objects, global mutable state, speculative abstraction, service locators and designs that hide business invariants.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Service calls GlobalContext.get("gateway") everywhere
   Dependency and lifetime become implicit
```

IN PLAIN ENGLISH

Explain the observable cost and a focused repair: constructor injection for hidden dependencies, cohesive services for mixed responsibilities, or simpler code for unnecessary layers. Avoid naming smells without showing actual impact.

INTERVIEW TRAP

A long class is not automatically wrong; incoherent change responsibilities matter.

FOLLOW-UP EXPLAINED

What concrete bug or change cost supports the refactor?
Answer: Show an observable defect: duplicate payment after a retry, leaked subscription, shared mutable state, or a new feature requiring edits to many unrelated classes.

Q100 ADVANCED

What makes an architect-level pattern answer strong?

THE SHORT ANSWER

State the problem, required variation, chosen collaboration, tradeoff, failure boundary and verification.

VISUAL MEMORY MAP



SOURCE EXAMPLE / JAVA IDEA

```
Problem: interchangeable tax policy
   Choice: Strategy + explicit registry
   Evidence: shared contract and rounding/unknown-code tests
```

IN PLAIN ENGLISH

Use a short diagram and one realistic example. Explain a simpler alternative and why it fits or does not fit. Connect design structure to correctness and maintenance rather than reciting a catalog definition.

INTERVIEW TRAP

A pattern name without context is not a design decision.

FOLLOW-UP EXPLAINED

What change would make you remove the pattern?
Answer: Remove it when the expected variation disappears, a plain function or record is clearer, or the abstraction hides invariants and increases defect or review cost.

WORKED CODING LABORATORY

12 runnable Java 17 exercises

Every lab is a complete self-contained class using the Java standard library. Compile all sources with javac --release 17, then execute Lab01 through Lab12. They passed the source-provided main-method checks.

C01	Strategy with a validated registry
C02	Factory Method in a creator workflow
C03	Builder with defensive copying
C04	Adapter and decorator order
C05	Composite with immutable children
C06	State with explicit legal actions
C07	Template Method with cleanup
C08	Observer with subscription lifetime
C09	Command with a local undo contract
C10	Chain with deliberate rejection
C11	Visitor with explicit double dispatch
C12	Memento with opaque immutable state

Strategy with a validated registry

TASK

Select an algorithm explicitly and reject unsupported policy identifiers.

COMPLETE JAVA 17 SOURCE

```
import java.util.Map;
public class Lab01 {
  interface Discount { long apply(long cents); }
  static long price(String code, long cents) {
    if (cents < 0) throw new IllegalArgumentException("Negative amount");
    Map<String,Discount> policies = Map.of(
        "NONE", c -> c, "TEN", c -> c - c / 10);
    Discount d = policies.get(code);
    if (d == null) throw new IllegalArgumentException("Unknown policy");
    return d.apply(cents);
  }
  public static void main(String[] args) {
    if (price("TEN", 1000) != 900) throw new AssertionError();
    if (price("NONE", 1000) != 1000) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

Save as Lab01.java. Compile/run with a Java 17+ JDK. The example uses integer cents and rounds the discount down by integer division. Add boundary checks for zero, small amounts, unknown code and negative amounts. The registry is rebuilt per call only to keep this small example self-contained; a real immutable registry can be assembled once. Explain when a two-branch switch would remain simpler.

Factory Method in a creator workflow

TASK

Let subclasses supply one product while preserving a shared workflow.

COMPLETE JAVA 17 SOURCE

```
public class Lab02 {
  interface Parser { String parse(String input); }
  abstract static class Job {
    protected abstract Parser createParser();
    final String run(String input) { return createParser().parse(input); }
  }
  static final class TrimJob extends Job {
    protected Parser createParser() { return String::trim; }
  }
  static final class UpperJob extends Job {
    protected Parser createParser() {
      return s -> s.toUpperCase(java.util.Locale.ROOT);
    }
  }
  public static void main(String[] args) {
    if (!new TrimJob().run(" x ").equals("x")) throw new AssertionError();
    if (!new UpperJob().run("x").equals("X")) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

The overridable createParser method is the creation variation point; run is the stable template. The parser contract here is deliberately tiny and does not accept null. Add contract cases for both products, then refactor the creator to accept a Parser or Supplier and discuss whether inheritance is justified. Do not call a static switch alone the same GoF collaboration.

Builder with defensive copying

TASK

Build an immutable value without exposing the builder's mutable list.

COMPLETE JAVA 17 SOURCE

```
import java.util.ArrayList;
import java.util.List;
public class Lab03 {
  record Report(String title, List<String> sections) {
    Report {
      if (title == null || title.isBlank())
        throw new IllegalArgumentException("Title required");
      sections = List.copyOf(sections);
    }
  }
  static final class Builder {
    private final String title;
    private final List<String> sections = new ArrayList<>();
    Builder(String title) { this.title = title; }
    Builder section(String s) { sections.add(s); return this; }
    Report build() { return new Report(title, sections); }
  }
  public static void main(String[] args) {
    Builder b = new Builder("Interview").section("Strategy");
    Report r = b.build(); b.section("State");
    if (r.sections().size() != 1) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

The record canonical constructor reassigns its parameter to a copied immutable list. String elements are immutable; mutable section objects would need a deeper policy. Test missing title, null element and attempted list mutation. Explain why final references alone would not be enough and why the builder is not intended for concurrent sharing.

Adapter and decorator order

TASK

Translate a vendor contract, then wrap the adapted operation with explicit behavior.

COMPLETE JAVA 17 SOURCE

```
import java.util.ArrayList;
import java.util.List;
public class Lab04 {
  interface Sender { void send(String text); }
  static final class Vendor {
    final List<String> received = new ArrayList<>();
    void deliver(byte[] bytes) {
      received.add(new String(bytes, java.nio.charset.StandardCharsets.UTF_8));
    }
  }
  static Sender adapt(Vendor v) {
    return text -> v.deliver(text.getBytes(java.nio.charset.StandardCharsets.UTF_8));
  }
  static Sender upper(Sender next) {
    return text -> next.send(text.toUpperCase(java.util.Locale.ROOT));
  }
  static Sender prefix(Sender next) { return text -> next.send("tag:" + text); }
  public static void main(String[] args) {
    Vendor v = new Vendor();
    upper(prefix(adapt(v))).send("x");
    prefix(upper(adapt(v))).send("x");
    if (!v.received.equals(List.of("tag:X", "TAG:X")))
      throw new AssertionError(v.received);
  }
}
```

VERIFY + DISCUSS

adapt translates String to the vendor byte API. upper and prefix are same-contract wrappers; their order changes which text gets transformed. Explain the actual order from outermost to innermost. Add a throwing vendor case and define propagation rather than swallowing failures. Real adapter contracts may require unit, authentication and uncertain-outcome translation beyond encoding.

Composite with immutable children

TASK

Price leaves and groups through one component contract.

COMPLETE JAVA 17 SOURCE

```
import java.util.List;
public class Lab05 {
  interface Item { long cents(); }
  record Product(long cents) implements Item {
    Product { if (cents < 0) throw new IllegalArgumentException(); }
  }
  record Bundle(List<Item> children) implements Item {
    Bundle { children = List.copyOf(children); }
    public long cents() {
      long total = 0;
      for (Item i : children) total = Math.addExact(total, i.cents());
      return total;
    }
  }
  public static void main(String[] args) {
    Item basket = new Bundle(List.of(new Product(100),
        new Bundle(List.of(new Product(50), new Product(25)))));
    if (basket.cents() != 175) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

Both Product and Bundle satisfy Item. List copying and immutable record examples simplify ownership. Math.addExact detects overflow instead of silently corrupting totals. Test empty groups, nested groups and overflow. Very deep trees can still exceed the stack. A mutable graph version would need a cycle/ownership policy that this fixed example avoids.

State with explicit legal actions

TASK

Reject an invalid transition while keeping state-specific behavior visible.

COMPLETE JAVA 17 SOURCE

```
public class Lab06 {
  interface State { State pay(); State ship(); String name(); }
  record Pending() implements State {
    public State pay() { return new Paid(); }
    public State ship() { throw new IllegalStateException("Not paid"); }
    public String name() { return "PENDING"; }
  }
  record Paid() implements State {
    public State pay() { return this; } // explicit idempotent local policy
    public State ship() { return new Shipped(); }
    public String name() { return "PAID"; }
  }
  record Shipped() implements State {
    public State pay() { throw new IllegalStateException("Already shipped"); }
    public State ship() { return this; }
    public String name() { return "SHIPPED"; }
  }
  public static void main(String[] args) {
    State s = new Pending(); s = s.pay(); s = s.ship();
    if (!s.name().equals("SHIPPED")) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

This is an in-memory state model: pay does not actually charge a provider. The context would own the current State reference. Add invalid-action checks and compare a small transition table. For a persisted order, enforce expected state/version at the durable store and deduplicate event identity. Returning this for repeated pay is not a general guarantee of idempotent external payment.

Template Method with cleanup

TASK

Keep execution sequence fixed and release acquired resources after failure.

COMPLETE JAVA 17 SOURCE

```
import java.util.ArrayList;
import java.util.List;
public class Lab07 {
  abstract static class ImportJob {
    final void run() {
      open();
      try { read(); validate(); save(); }
      finally { close(); }
    }
    abstract void open(); abstract void read();
    abstract void validate(); abstract void save(); abstract void close();
  }
  public static void main(String[] args) {
    List<String> calls = new ArrayList<>();
    ImportJob j = new ImportJob() {
      void open() { calls.add("open"); }
      void read() { calls.add("read"); }
      void validate() { calls.add("validate"); }
      void save() { calls.add("save"); }
      void close() { calls.add("close"); }
    };
    j.run();
    if (!calls.equals(List.of("open","read","validate","save","close")))
      throw new AssertionError(calls);
  }
}
```

VERIFY + DISCUSS

If read, validate or save throws, close still runs. open sits before try under the explicit assumption it either succeeds completely or cleans up its own partial acquisition. Test each failure point. In real resource code, try-with-resources can preserve suppressed cleanup exceptions more clearly; this simple close contract must be reviewed so cleanup does not mask the original error.

Observer with subscription lifetime

TASK

Notify a snapshot of listeners and allow explicit removal.

COMPLETE JAVA 17 SOURCE

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;
public class Lab08 {
  static final class Events {
    private final List<Consumer<String>> listeners = new ArrayList<>();
    Runnable subscribe(Consumer<String> l) {
      listeners.add(l); return () -> listeners.remove(l);
    }
    void publish(String e) {
      for (Consumer<String> l : List.copyOf(listeners)) l.accept(e);
    }
  }
  public static void main(String[] args) {
    Events events = new Events(); List<String> seen = new ArrayList<>();
    Runnable unsubscribe = events.subscribe(seen::add);
    events.publish("placed"); unsubscribe.run(); events.publish("paid");
    if (!seen.equals(List.of("placed"))) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

This single-threaded example uses fail-fast callback exception propagation: one throwing callback stops later callbacks for that publication. Snapshotting tolerates listener-list changes within a callback, but does not make subscription/publishing thread-safe. Test unsubscribe, nested publication and a throwing listener. Durable notification requires more than this in-memory observer arrangement.

Command with a local undo contract

TASK

Represent an edit as an object and restore the captured prior value.

COMPLETE JAVA 17 SOURCE

```
public class Lab09 {
  static final class Editor { String title = "old"; }
  static final class Rename {
    private final Editor editor; private final String next;
    private String previous; private boolean executed;
    Rename(Editor e, String n) { editor = e; next = n; }
    void execute() {
      if (executed) throw new IllegalStateException("Already executed");
      previous = editor.title; editor.title = next; executed = true;
    }
    void undo() {
      if (!executed) throw new IllegalStateException("Nothing to undo");
      editor.title = previous; executed = false;
    }
  }
  public static void main(String[] args) {
    Editor e = new Editor(); Rename c = new Rename(e, "new");
    c.execute(); c.undo();
    if (!e.title.equals("old")) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

This local command assumes a sequential owner-controlled edit history with no competing edits between execution and undo. Test undo-before-execute and double execution. Discuss how a version check/history manager would protect newer edits. Do not serialize the live Editor reference and call it a durable distributed command; store intent and identity for that separate use case.

Chain with deliberate rejection

TASK

Apply ordered checks and stop on the first rejection.

COMPLETE JAVA 17 SOURCE

```
import java.util.List;
import java.util.Optional;
public class Lab10 {
  record Request(boolean authenticated, int units) {}
  interface Check { Optional<String> reject(Request r); }
  static Optional<String> check(Request r, List<Check> checks) {
    for (Check c : checks) {
      Optional<String> rejection = c.reject(r);
      if (rejection.isPresent()) return rejection;
    }
    return Optional.empty();
  }
  public static void main(String[] args) {
    Check auth = r -> r.authenticated() ? Optional.empty()
        : Optional.of("unauthenticated");
    Check quota = r -> r.units() <= 10 ? Optional.empty()
        : Optional.of("quota");
    if (!check(new Request(false,20), List.of(auth,quota))
        .equals(Optional.of("unauthenticated"))) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

This validation-chain variant executes all checks until rejection, rather than the classic first-handler-wins request model. After no rejection, a separate terminal action processes the request. Validate nonnegative units at the input boundary. Test order, early termination and thrown exceptions. Define an explicit fail-closed policy where a check's infrastructure failure must not permit access.

Visitor with explicit double dispatch

TASK

Add printing and evaluation over the same expression hierarchy.

COMPLETE JAVA 17 SOURCE

```
public class Lab11 {
  interface Expr { <R> R accept(Visitor<R> v); }
  interface Visitor<R> { R number(Num n); R add(Add a); }
  record Num(int value) implements Expr {
    public <R> R accept(Visitor<R> v) { return v.number(this); }
  }
  record Add(Expr left, Expr right) implements Expr {
    public <R> R accept(Visitor<R> v) { return v.add(this); }
  }
  static final class Eval implements Visitor<Integer> {
    public Integer number(Num n) { return n.value(); }
    public Integer add(Add a) {
      return Math.addExact(a.left().accept(this), a.right().accept(this));
    }
  }
  static final class Print implements Visitor<String> {
    public String number(Num n) { return Integer.toString(n.value()); }
    public String add(Add a) {
      return "(" + a.left().accept(this) + "+" + a.right().accept(this) + ")";
    }
  }
  public static void main(String[] args) {
    Expr e = new Add(new Num(2), new Num(3));
    if (e.accept(new Eval()) != 5) throw new AssertionError();
    if (!e.accept(new Print()).equals("(2+3)")) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

accept chooses the type-specific visitor method from inside the concrete element, then the visitor operation dispatches dynamically. Add another visitor without changing Expr implementations. Then add Multiply and observe how every visitor contract/implementation must evolve. Validate null children in production constructors and bound traversal depth for untrusted trees.

Memento with opaque immutable state

TASK

Capture editor state and restore it without caretaker mutation.

COMPLETE JAVA 17 SOURCE

```
public class Lab12 {
  static final class Editor {
    private String text = "";
    static final class Snapshot {
      private final String text;
      private Snapshot(String text) { this.text = text; }
    }
    void write(String value) { text = value; }
    String text() { return text; }
    Snapshot save() { return new Snapshot(text); }
    void restore(Snapshot s) { text = s.text; }
  }
  public static void main(String[] args) {
    Editor e = new Editor(); e.write("first");
    Editor.Snapshot before = e.save(); e.write("second");
    e.restore(before);
    if (!e.text().equals("first")) throw new AssertionError();
  }
}
```

VERIFY + DISCUSS

The caretaker can hold Snapshot but receives no accessor or mutation API in this example; Editor owns restoration semantics. String state is immutable. Test multiple snapshots and explain how richer mutable graphs require copying/shared-immutable structure. Java nestmate access means this teaching top-level class can technically access nested private members; the design boundary is intended as an external caretaker API, not a security boundary.

CASE STUDY / ORDER PLACEMENT

Small patterns. Strong guarantees.

Scenario from the source guide: place an order, choose a payment provider, record the accepted result and notify the customer. A rejection differs from a timeout, and duplicate requests must not create a second charge.

FACADE-STYLE ENTRY

OrderApplicationService coordinates the use case; it does not make remote operations atomic.

STRATEGY + ADAPTER

A chosen PaymentPort implementation routes to a provider SDK; the adapter translates uncertain outcomes.

STATE TRANSITION

Store order status and version atomically in the authoritative database.

COMMAND + OUTBOX

Persist notification intent in the same local transaction; deliver and deduplicate asynchronously.

NEGATIVE TESTS

Reject replayed IDs, concurrent approvals, invalid transitions, and ambiguous payment outcomes.

TWO MOCK INTERVIEWS

Speak and defend each pattern

ROUND A - IMPLEMENTATION

Explain Strategy, Factory Method, Builder and Adapter. Implement one variation point. Identify which contract you test.

ROUND B - SENIOR DESIGN

Design order checkout with multiple providers, concurrency, retries, durable event delivery and safe recovery. Defend what a pattern cannot guarantee.

MARK YOUR ANSWER

Problem -> roles -> tiny example -> cost -> failure path -> test. Keep the spoken answer short, then expand when questioned.

Readiness check: run the 12 labs from a clean folder, compare all 23 intents, and retake every follow-up you initially missed.